

DriveU

Overview:

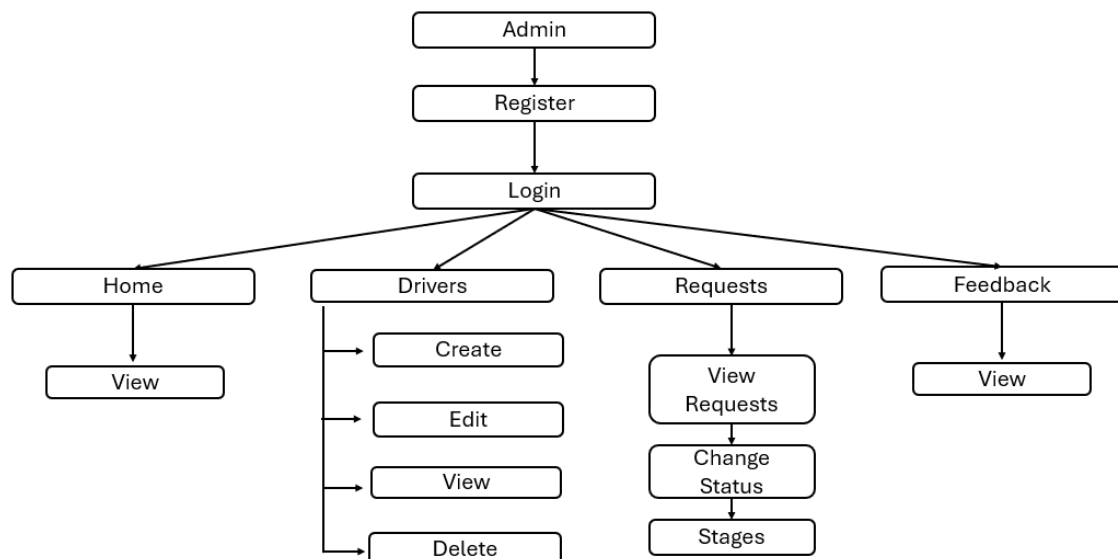
DriveU is an all-in-one platform designed to simplify the process of booking professional drivers for diverse transportation needs. Whether it's daily commutes, outstation trips, or special events, it connects users with experienced and verified drivers. The platform ensures convenience, safety, and affordability, allowing users to book drivers on-demand or schedule them in advance. DriveU aims to enhance the travel experience by offering reliable services tailored to individual preferences.

Users of the System:

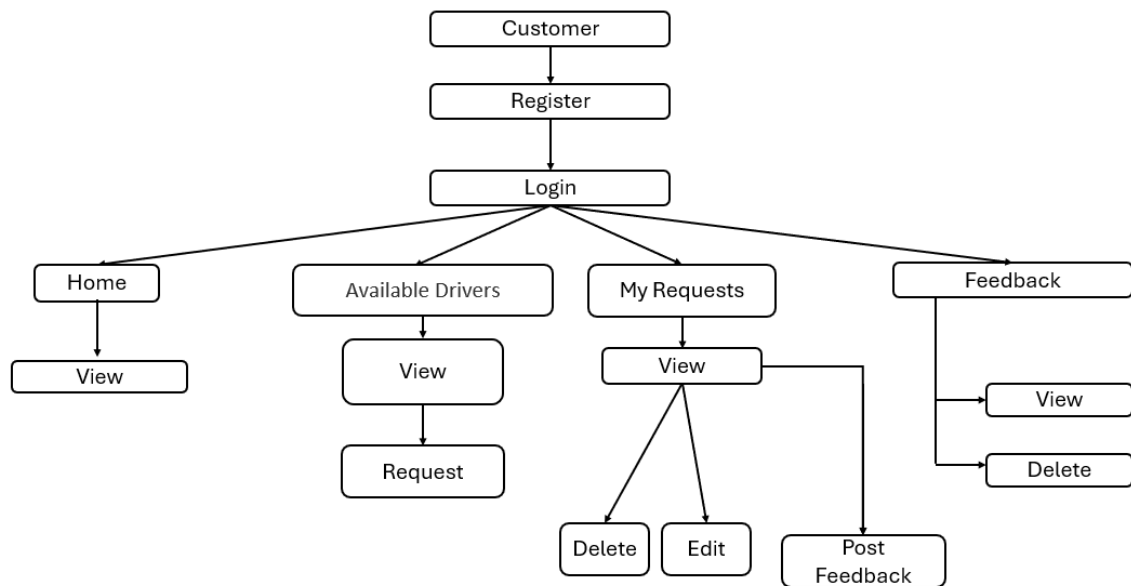
1. Admin
2. Customer

Admin Actions:

Flow Diagram:



Customer Actions:
Flow Diagram:



Modules of the Application:

Admin:

1. Register
2. Login
3. Home
4. Drivers
 - o Add Driver
 - o View Drivers
5. Requests
 - o View Requests
6. Feedback
 - o View Feedback
7. Logout

Customer:

1. Register
2. Login
3. Home
4. Available Drivers
5. My Requests
6. Feedback
7. Logout

Technology Stack

Front End	Angular 10+, HTML, CSS
Back End	Java 17, Spring boot 3.0.1, Mysql
Authentication	JWT for User Authentication.

Application assumptions:

1. The login page should be the first page rendered when the application loads.
2. Manual routing should be restricted by implementing an Auth Guard using the canActivate interface. For instance, if a user manually enters an unauthorized link, they should be redirected to the login page instead of navigating to the requested page.
3. Unless logged into the system, the user cannot navigate to any other pages.
4. Logging out must again redirect to the login page.

Backend Endpoints:

Controller:

AuthController API Endpoints

Action	URL	Method	Role	Response
User Registration	/api/register	POST	All	Return response code 201 with the User object on success, 401 for authentication failure, and 409 Returns "A user with this email already exists" if email is duplicate.
User Login	/api/login	POST	All	Return response code 201 with LoginDTO object with token, role, user ID, and username on success. Return res 401 (Unauthorized): Returns error for invalid credentials.

DriverController API Endpoints

Action	URL	Method	Role	Response
Add Driver	/api/driver	POST	Admin	Returns response code 201 with Driver object on successful creation or 401 and 409 if failure
View Driver by ID	/api/driver/{driverId}	GET	Admin	Returns response code 200 with Driver object on successful retrieval , 204 for no content or 404 if not found.
View All Drivers	/api/driver	GET	ALL	Returns response code 200 with a list of Driver objects on successful retrieval, 204 for no content or 400 if failure.
Update Driver	/api/driver/{driverId}	PUT	Admin	Returns response code 200 with updated Driver object on successful update or 404 if not found.
Delete Driver	/api/driver/{driverId}	DELETE	Admin	Returns response code 200 with deleted Driver

				object on successful deletion or 404 if not found.
--	--	--	--	--

DriverRequestController API Endpoints

Action	URL	Method	Role	Response
Add Driver Request	/api/driverRequest	POST	Customer	Returns response code 201 with DriverRequest object on successful creation or 401 and 409 if failure
View Driver Request by ID	/api/driverRequest/{driverRequestId}	GET	Admin, Customer	Returns response code 200 with DriverRequest object on successful retrieval , 204 for no content or 404 if not found.
View Driver Requests by User ID	/api/driverRequest/user/{userId}	GET	Customer	Returns response code 200 with a list of DriverRequest objects on successful retrieval or 404 if failure.

View All Driver Requests	/api/driverRequest	GET	Admin	Returns response code 200 with a list of DriverRequest objects on successful retrieval, 204 for no content or 400 if failure.
Update Driver Request	/api/driverRequest/{driverRequestId}	PUT	Admin , Customer	Returns response code 200 with updated DriverRequest object on successful update or 404 if not found.
View Driver Requests by Driver ID	/api/driverRequest/driver/{driverId}	GET	Admin	Returns response code 200 with a list of DriverRequest objects for the specified driver or or 404 if failure.
Delete Driver Request	/api/driverRequest/{driverRequestId}	DELETE	Customer	Returns response code 200 on successful deletion or 404 if the request is not found.

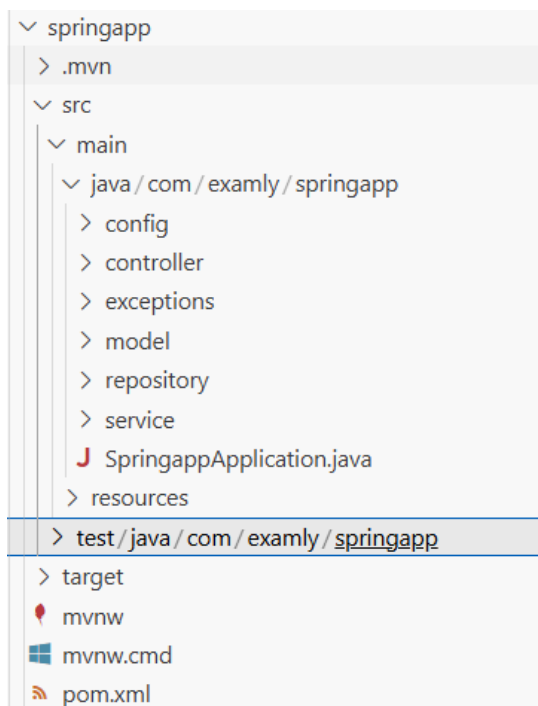
FeedbackController API Endpoints

Action	URL	Method	Role	Response
Create Feedback	/api/feedback	POST	Customer	Returns response code 201 with Feedback object on successful creation or else 401 and 409 if failure
View Feedback by Id	/api/feedback/{feedbackId}	GET	Admin, Customer	Returns response code 200 with Feedback object on successful retrieval, 204 for no content or else 404 if failure
View All Feedbacks	/api/feedback	GET	ALL	Returns response code 200 with List<Feedback> object on successful retrieval , 204 for no content or else 400 if failure
View Feedbacks by User	/api/feedback/user/{userId}	GET	Customer	Returns response code 200 with List<Feedback> object on successful retrieval, 204 for no content or else 404 if failure
Delete Feedback	/api/feedback/{feedbackId}	DELETE	Customer	Returns response code

				200 with deleted Feedback object within response body on successful deletion or else 404 if failure
--	--	--	--	---

Backend Folder Structure:

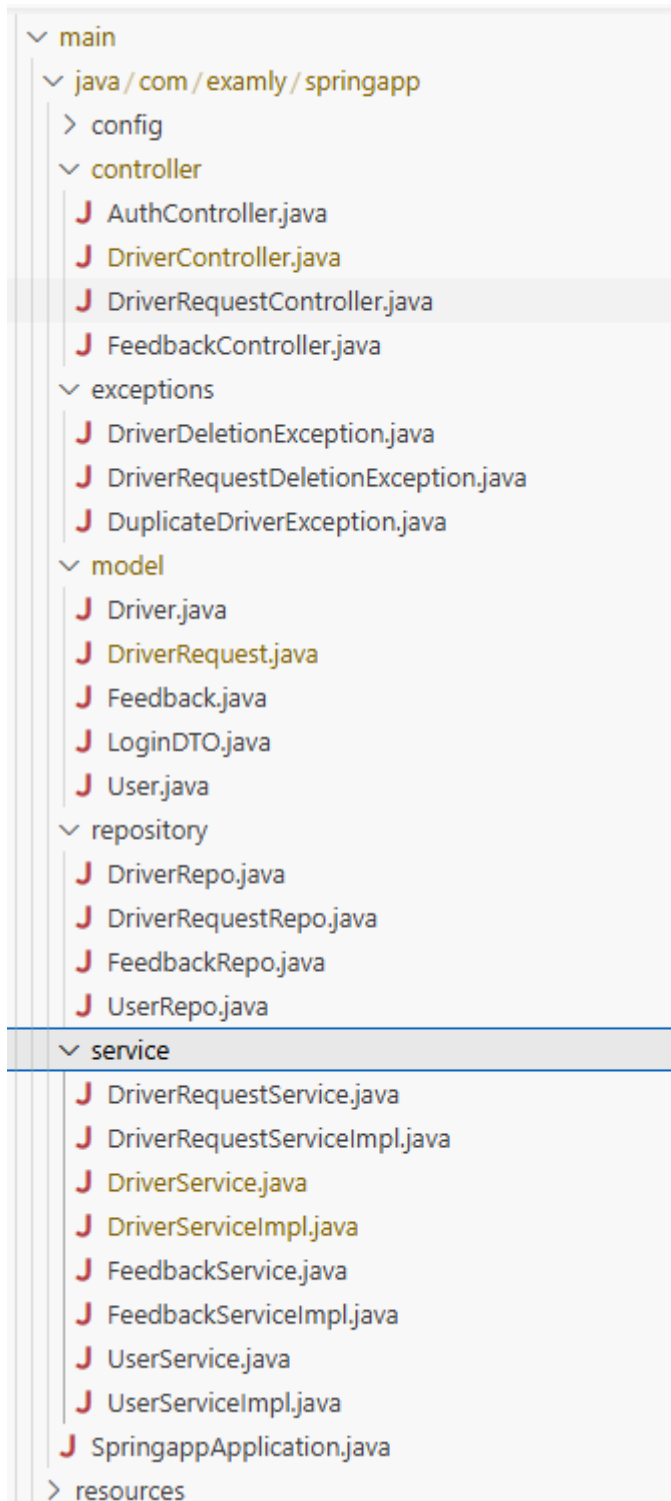
Create folders named as model, repository, service, controller and config inside springapp as mentioned in the below screenshot.

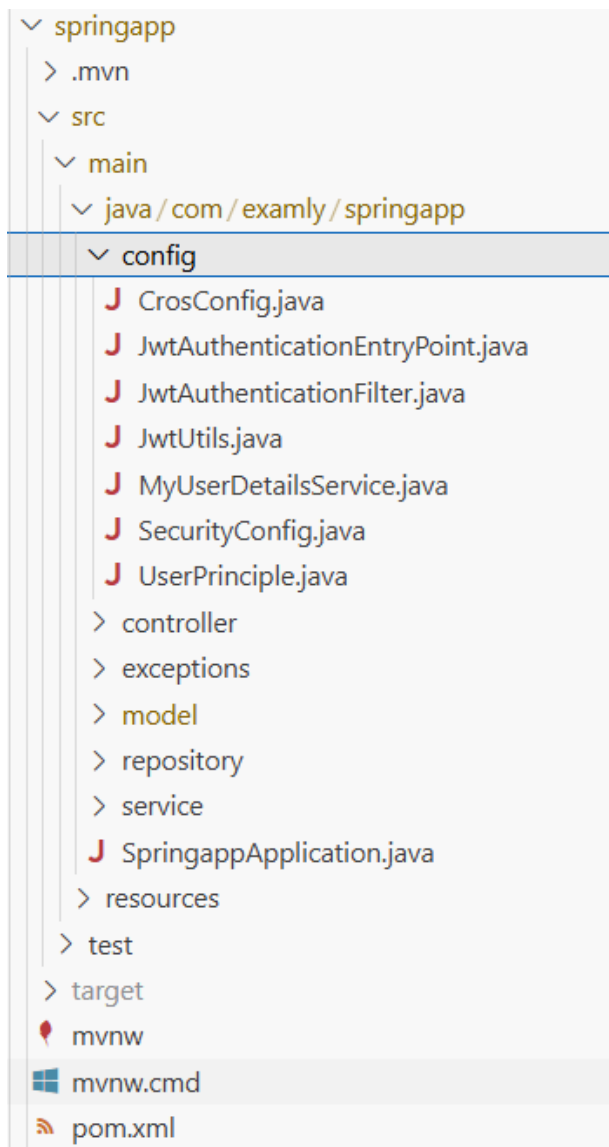


In config folder, create a class `CrosConfig` with `@Configuration` annotation which implements `WebMvcConfigurer` to enable CORS functionality and also add all files related to JWT.

- In controller folder, create classes named **AuthController**, **DriverController**, **DriverRequestController**, **FeedbackController** implement REST APIs (Refer the API table).
- In repository folder, create the interfaces named as **UserRepo**, **FeedbackRepo**, **DriverRepo**, **DriverRequestRepo** which extends **JpaRepository**.

- Ensure adherence to the specified API outlined in the API table, ensuring consistency in class names, variables, and folder structure as detailed.





Model Classes:

Create model folder. Inside the model folder create all the classes mentioned below.

User

This class stores all user information.

- **userId:** Long (auto-generated, primary key)
- **email:** String
- **password:** String
- **username:** String
- **mobileNumber:** String
- **userRole:** String (Admin/Customer)

Driver Class

This class stores all information related to drivers.

- **driverId:** Long (auto-generated, primary key).
- **driverName:** String (Name of the driver).
- **licenseNumber:** String (License number of the driver).
- **experienceYears:** Integer (Years of driving experience).

- **contactNumber:** String (Contact number of the driver).
- **availabilityStatus:** String (Driver's availability status; allowed values: "Active", "Inactive", "On Leave").
- **address:** String (Residential address of the driver).
- **vehicleType:** String (Type of vehicle assigned to the driver, e.g., "Sedan", "SUV", "Bike").
- **hourlyRate:** Double (Hourly rate charged by the driver).
- **image:** String (Base64-encoded image of the driver).
 - o Marked with @Lob and stored as a LONGBLOB.
 - o Nullable: true.

DriverRequest Class

This class stores all information related to driver requests.

- **driverRequestId:** Long (auto-generated, primary key).
- **user:** User (The user who made the request).
 - o @ManyToOne, @JoinColumn(name = "userId", nullable = false).
- **driver:** Driver (The assigned driver for the request).
 - o @ManyToOne, @JoinColumn(name = "driverId", nullable = true).
- **requestDate:** LocalDate (Date when the request was created).
- **status:** String (Status of the request; allowed values are "Pending", "Approved", "Rejected", "Trip End", "Closed").
- **tripDate:** LocalDate (Scheduled date for the trip).
- **timeSlot:** LocalTime (Time for the trip, e.g., 10:30 AM).
- **pickupLocation:** String (Starting location of the trip).
- **dropLocation:** String (Destination location of the trip).
- **estimatedDuration:** String (Expected duration of the trip, e.g., "3 hours").
- **paymentAmount:** Double (Amount charged for the trip).
 - o Nullable: true.
- **comments:** String (Additional notes or comments).
- **actualDropTime:** LocalTime (Actual time when the trip ended).
 - o Nullable: true.
- **actualDropDate:** LocalDate (Actual date when the trip ended).
 - o Nullable: true.
- **actualDuration:** String (Actual duration of the trip).
 - o Nullable: true.

Feedback Class

This class stores all feedback-related information.

- **feedbackId:** Long (auto-generated, primary key).
- **feedbackText:** String (textual feedback provided by the user).
- **date:** LocalDate (date the feedback was submitted).
- **user:** User (@ManyToOne, @JoinColumn(name = "userId", nullable = false)) - the user providing the feedback.
- **driver:** Driver (@ManyToOne, @JoinColumn(name = "driverId", nullable = true)) - the driver being reviewed (nullable).
- **category:** String (e.g., "Driver Performance", "Service Experience").
- **rating:** Integer (rating from 1 to 5).

LoginDTO Class

- **token:** String
- **username:** String
- **userRole:** String
- **userId:** Long

DTO to be used:

LoginDTO class

- The LoginDTO (Data Transfer Object) in the backend is a simple object used to transfer data during the user login process. It serves as an intermediary between the frontend (client-side) and the backend (server-side) by holding the user's credentials, typically the username (or email) and password, that are submitted during login.

Exceptions to be used:

The **exceptions** folder in the com.examly.springapp package contains all custom exception classes used to handle specific runtime errors within the application. These exceptions extend RuntimeException, allowing the application to provide meaningful error messages and handle errors gracefully during execution. Below is an overview of the exceptions included in this folder:

DriverDeletionException

This exception is thrown when an error occurs during the deletion of a driver.

- **Purpose:** To handle cases where the deletion of a driver fails due to system constraints, dependencies, or other restrictions.
- **Constructor:** Accepts a custom error message describing the reason for the exception.

DuplicateDriverException

This exception is thrown when attempting to add a new driver with duplicate or conflicting information.

- **Purpose:** To prevent the creation of drivers with duplicate attributes, such as license numbers or IDs, ensuring data consistency.
- **Constructor:** Accepts a custom error message indicating the nature of the conflict.

DriverRequestDeletionException

This exception is thrown when an error occurs during the deletion of a driver request.

- **Purpose:** To handle scenarios where a driver request cannot be deleted due to system constraints or dependencies, such as ongoing or unresolved trips.
- **Constructor:** Accepts a custom error message explaining the reason for the failure.

Interfaces to be used:

UserService Interface

Create an interface **UserService** inside the service folder with the below-mentioned abstract methods and provide implementation for these methods in **UserServiceImpl** class.

- **createUser(User user):** Creates a new user.
- **loginUser(User user):** Logs in a user.

DriverService Interface

Create an interface **DriverService** inside the service folder with the below-mentioned abstract methods and provide implementation for these methods in the **DriverServiceImpl** class.

- **addDriver(Driver driver)**: Adds a new driver to the system. This method accepts a Driver object and handles the logic for persisting it in the database.
- **getDriverById(Long driverId)**: Retrieves a driver by the specified driver ID. It returns an Optional<Driver> that may contain the corresponding driver.
- **getAllDrivers()**: Retrieves a list of all drivers available in the system. This method returns a collection of Driver objects.
- **updateDriver(Long driverId, Driver driver)**: Updates the driver identified by the provided driver ID. It accepts a Driver object with updated details to be persisted.
- **deleteDriver(Long driverId)**: Deletes a driver identified by the specified driver ID. This method returns the Driver object that has been deleted.

DriverRequestService Interface

Create an interface **DriverRequestService** inside the service folder with the below-mentioned abstract methods and provide implementation for these methods in the **DriverRequestServiceImpl** class.

- **addDriverRequest(DriverRequest driverRequest)**: Adds a new driver request to the system. This method accepts a DriverRequest object and handles the logic for persisting it in the database.
- **getDriverRequestById(Long driverRequestId)**: Retrieves a driver request by the specified request ID. It returns an Optional<DriverRequest> that may contain the corresponding request.
- **getAllDriverRequests()**: Retrieves a list of all driver requests in the system. This method returns a collection of DriverRequest objects.
- **updateDriverRequest(Long driverRequestId, DriverRequest driverRequest)**: Updates the driver request identified by the provided request ID. It accepts a DriverRequest object with updated details to be persisted.
- **deleteDriverRequest(Long driverRequestId)**: Deletes a driver request identified by the specified request ID. This method returns the DriverRequest object that has been deleted.
- **findDriverRequestsByUserId(Long userId)**: Retrieves a list of all driver requests submitted by a specific user. This method returns a collection of DriverRequest objects associated with the provided user ID.
- **findDriverRequestsByDriverId(Long driverId)**: Retrieves a list of all driver requests assigned to a specific driver. This method returns a collection of DriverRequest objects associated with the provided driver ID.

FeedbackService Interface

Create an interface **FeedbackService** inside the service folder with the below-mentioned abstract methods and provide implementation for these methods in the **FeedbackServiceImpl** class.

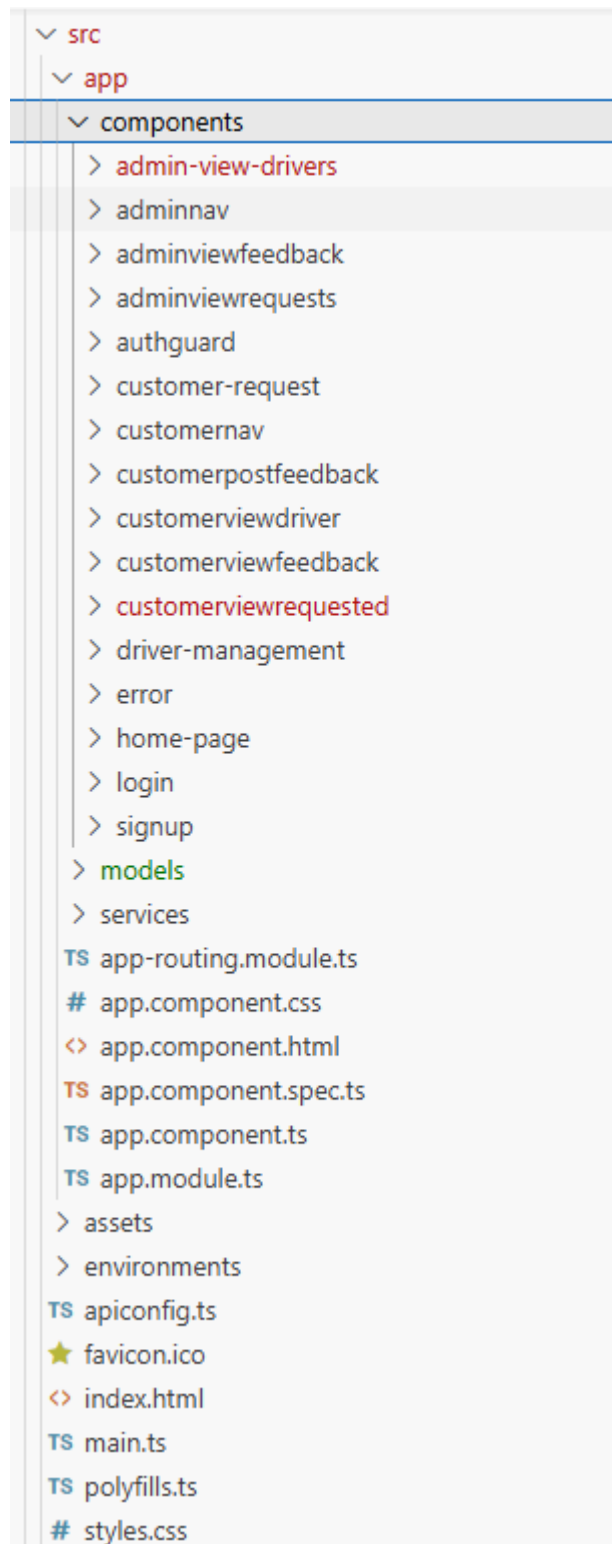
- **createFeedback(Feedback feedback)**: Creates a new feedback entry in the system. This method accepts a Feedback object and handles the logic for persisting it in the database.
- **getFeedbackById(Long feedbackId)**: Retrieves feedback details by the specified feedback ID. It returns a Feedback object with the corresponding feedback details.
- **getAllFeedbacks()**: Retrieves a list of all feedbacks available in the system. This method returns a collection of Feedback objects.
- **deleteFeedback(Long feedbackId)**: Deletes feedback information by the specified feedback ID. This method returns the Feedback object that has been deleted.

- **getFeedbacksByUserId(Long userId):** Retrieves a list of all feedback entries submitted by a specific user. This method returns a collection of Feedback objects associated with the provided user ID.

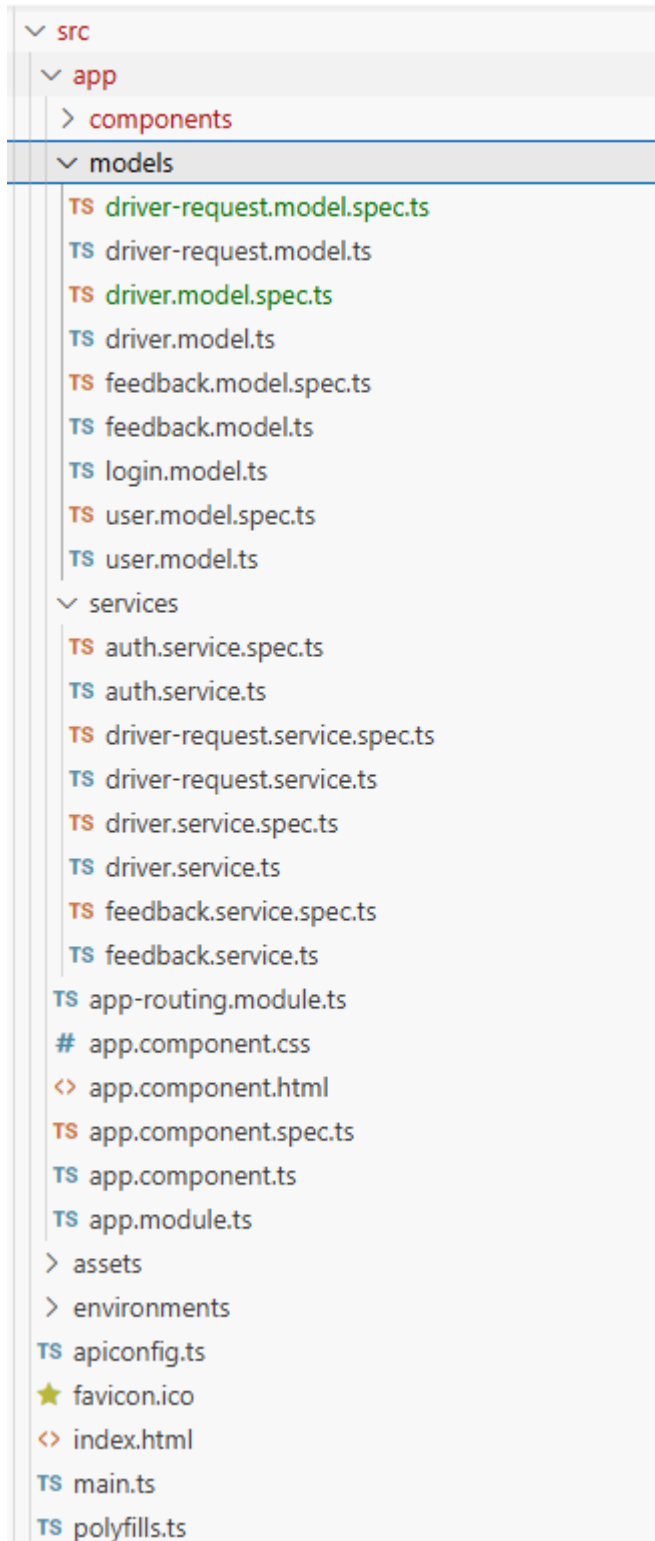
Frontend Requirements:

- Create a folder named components inside the app to store all the components. (Refer project structure screenshots).
- Create a folder named models inside app to store all the model interface.
- Create a folder named as services inside app to implement all the services.
- Create model interface referring the backend entities mentioned in the backend requirements accordingly.
- You can create your own components based on the application requirements.
- Import model files, services and components as required.

Project Folder Screenshot:



Services and Models:



Frontend Models:

User Model:

```
class User {  
  userId?: number;  
  email: string;  
  password: string;  
  username: string;  
  mobileNumber: string;  
  userRole: string;  
}
```

Login Model:

```
class Login {  
  email: string;  
  password: string;  
}
```

Driver Model:

```
interface Driver {  
  driverId?: number;  
  driverName: string;  
  licenseNumber: string;  
  experienceYears: number;  
  contactNumber: string;  
  availabilityStatus: string; //Initially, should be Active , Value should be "Active", "Inactive", or "On  
Leave" (no other values are allowed)  
  address: string;  
  vehicleType: string; // e.g., "Sedan", "SUV", "Bike"  
  hourlyRate: number;  
  image: string; // Base64-encoded image of the driver  
}
```

DriverRequest Model:

```
interface DriverRequest{  
  driverRequestId?: number;  
  userId: number;  
  driverId?: number;  
  requestDate: Date; // ISO Date format (YYYY-MM-DD)  
  status: string; // Status of the request; allowed values are "Pending", "Approved", "Rejected", "Trip  
End", "Closed"  
  tripDate: Date; // ISO Date format (YYYY-MM-DD)  
  timeSlot: Date; // Time as a Date object (compatible with LocalTime)  
  pickupLocation: string;  
  dropLocation: string;  
  estimatedDuration: string; // e.g., "3 hours"  
  paymentAmount?: number;  
  comments?: string;  
  actualDropTime?: Date;  
  actualDuration?: string;  
  actualDropDate?: Date;
```

```
}
```

Feedback Model:

```
class Feedback {  
  feedbackId?: number;  
  feedbackText: string;  
  date: Date;  
  userId: number;  
  driverId?: number; // Optional for non-driver feedback  
  category: string;  
  rating: number;  
}
```

Frontend services:

- Declare a public property `apiUrl` to store the backend URL in all the services.
- For example, `public apiUrl = 'http://localhost:8080'`. Instead of 'localhost', replace it with the URL of your workspace port 8080 URL.
- For the API's to be used please refer the API Table.
- Authorized token to be passed in headers for all end points.

1. AuthService(auth.service.ts):

Create a service name as **auth** inside `app/services` folder to implement the following functions.

Methods Overview:

- **register(user: User): Observable<any>:**
 - Use this method to register a new user. It sends a POST request to the `'/api/register'` endpoint with the user data provided as the body.
- **login(login : Login): Observable<any>:**
 - This method is used to authenticate a user by logging them in. It sends a POST request to the `'/api/login'` endpoint with the user's email and password. Upon successful login, it stores the JWT token in `localStorage` and updates the user's role and ID using `BehaviorSubjects`.

2. DriverService (driver.service.ts)

Create a service named `driver` inside `app/services` and implement the following functions.

Methods Overview

- **getAllDrivers(): Observable<Driver[]>**
 - Use this method to fetch all available drivers from the server.
 - It sends a **GET** request to the `/api/driver` endpoint with the authorization token prefixed with 'Bearer' stored in local storage.
- **getDriverById(driverId: number): Observable<Driver>**
 - Use this method to retrieve a specific driver by its ID.
 - It sends a **GET** request to the `/api/driver/:driverId` endpoint with the driver ID and the authorization token prefixed with 'Bearer' stored in local storage.
- **addDriver(driver: Driver): Observable<Driver>**
 - Call this method to add a new driver.

- o It sends a **POST** request to the `/api/driver` endpoint with the driver data provided as the request body and the authorization token prefixed with 'Bearer' stored in local storage.
- **updateDriver(driverId: number, driver: Driver): Observable<Driver>**
 - o Use this method to update an existing driver.
 - o It sends a **PUT** request to the `/api/driver/:driverId` endpoint with the driver ID and the updated driver data provided as the request body, along with the authorization token prefixed with 'Bearer' stored in local storage.
- **deleteDriver(driverId: number): Observable<void>**
 - o Call this method to delete a driver by its ID.
 - o It sends a **DELETE** request to the `/api/driver/:driverId` endpoint with the driver ID and the authorization token prefixed with 'Bearer' stored in local storage.

3. DriverRequestService (driver-request.service.ts)

Create a service named **driverRequest** inside `app/services` and implement the following functions.

Methods Overview

- **getAllDriverRequests(): Observable<DriverRequest[]>**
 - o Use this method to fetch all driver requests from the server.
 - o It sends a **GET** request to the `/api/driverRequest` endpoint with the authorization token prefixed with 'Bearer' stored in local storage.
- **getDriverRequestById(driverRequestId: number): Observable<DriverRequest>**
 - o Use this method to retrieve a specific driver request by its ID.
 - o It sends a **GET** request to the `/api/driverRequest/:driverRequestId` endpoint with the request ID and the authorization token prefixed with 'Bearer' stored in local storage.
- **getDriverRequestsByUserId(userId: number): Observable<DriverRequest[]>**
 - o Use this method to fetch all driver requests submitted by a specific user.
 - o It sends a **GET** request to the `/api/driverRequest/user/:userId` endpoint with the user ID and the authorization token prefixed with 'Bearer' stored in local storage.
- **getDriverRequestsByDriverId(driverId: number): Observable<DriverRequest[]>**
 - o Use this method to fetch all driver requests assigned to a specific driver.
 - o It sends a **GET** request to the `/api/driverRequest/driver/:driverId` endpoint with the driver ID and the authorization token prefixed with 'Bearer' stored in local storage.
- **addDriverRequest(driverRequest: DriverRequest): Observable<DriverRequest>**
 - o Call this method to add a new driver request.
 - o It sends a **POST** request to the `/api/driverRequest` endpoint with the request data provided as the request body and the authorization token prefixed with 'Bearer' stored in local storage.
- **updateDriverRequest(driverRequestId: number, driverRequest: DriverRequest): Observable<DriverRequest>**
 - o Use this method to update an existing driver request.
 - o It sends a **PUT** request to the `/api/driverRequest/:driverRequestId` endpoint with the request ID and the updated request data provided as the request body, along with the authorization token prefixed with 'Bearer' stored in local storage.

- **deleteDriverRequest(driverRequestId: number): Observable<void>**
 - o Call this method to delete a driver request by its ID.
 - o It sends a **DELETE** request to the `/api/driverRequest/:driverRequestId` endpoint with the request ID and the authorization token prefixed with 'Bearer' stored in local storage.

4. FeedbackService(feedback.service.ts):

- Create a service name as **feedback** inside `app/services` and implement the following functions in it.

Methods Overview:

- **sendFeedback(feedback: Feedback): Observable<Feedback>**
 - o Use this method to send feedback to the server. It sends a POST request to the `/api/feedback` endpoint with the feedback data provided and the authorization token prefixed with 'Bearer' stored in localStorage.
- **getAllFeedbacksByUserId(userId: number): Observable<Feedback[]>**
 - o This method retrieves all feedbacks submitted by a specific user. It sends a GET request to the `/api/feedback/user/:userId` endpoint with the user ID and the authorization token prefixed with 'Bearer' stored in localStorage.
- **deleteFeedback(feedbackId: number): Observable<void>**
 - o Call this method to delete a feedback with the specified ID. It sends a DELETE request to the `/api/feedback/:feedbackId` endpoint with the feedback ID and the authorization token prefixed with 'Bearer' stored in localStorage.
- **getFeedbacks(): Observable<Feedback[]>**
 - o This method fetches all feedbacks from the server. It sends a GET request to the `/api/feedback` endpoint with the authorization token prefixed with 'Bearer' stored in localStorage.

Validations:

Client-Side Validation:

- Implement client-side validation using HTML5 attributes and JavaScript to validate user input before making API requests.
- Provide immediate feedback to users for invalid input, such as displaying error messages near the input fields.

Server-Side Validation:

- Implement server-side validation in the controllers to ensure data integrity.
- Validate user input and API responses to prevent unexpected or malicious data from affecting the application.
- Return appropriate validation error messages to the user interface for any validation failures.

Exception Handling:

- Implement exception handling mechanisms in the controllers to gracefully handle errors and exceptions. Define custom exception classes for different error scenarios, such as API communication errors or database errors.

- Log exceptions for debugging purposes while presenting user-friendly error messages to users. Record all the exceptions and errors handled store in separate table “ErrorLogs”.

Error Pages:

Create custom error pages for different HTTP status codes (e.g., 404 Not Found, 500 Internal Server Error) to provide a consistent and user-friendly error experience. Ensure that error pages contain helpful information and guidance for users.

Thus, create a reliable and user-friendly web application that not only meets user expectations but also provides a robust and secure experience, even when faced with unexpected situations. Error page has to be displayed if something goes wrong.

Something Went Wrong

We're sorry, but an error occurred. Please try again later.

Frontend Screenshots:

- All the asterisk (*) marked fields are mandatory in the form. Make sure to mark all the field names with * symbol followed by the validations.

NavBar:

(adminnav component)

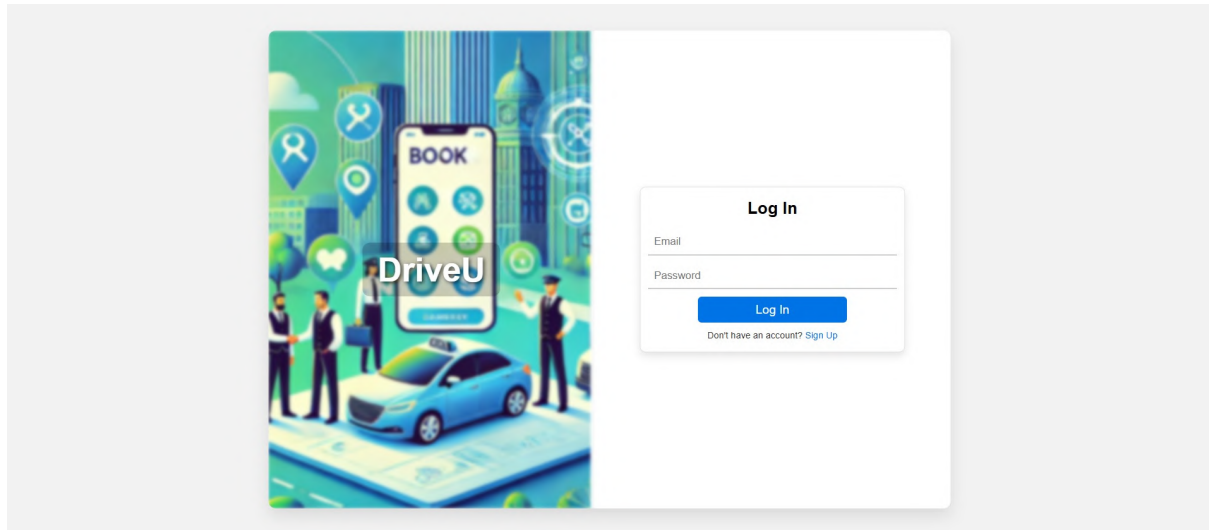


(customernav component)



Landing Page:

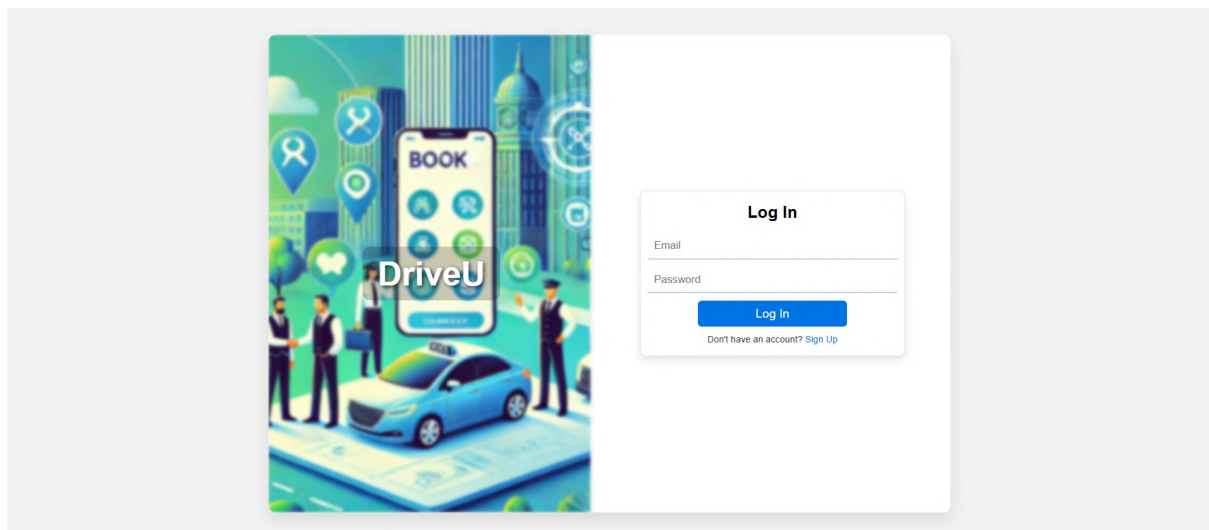
(login component)



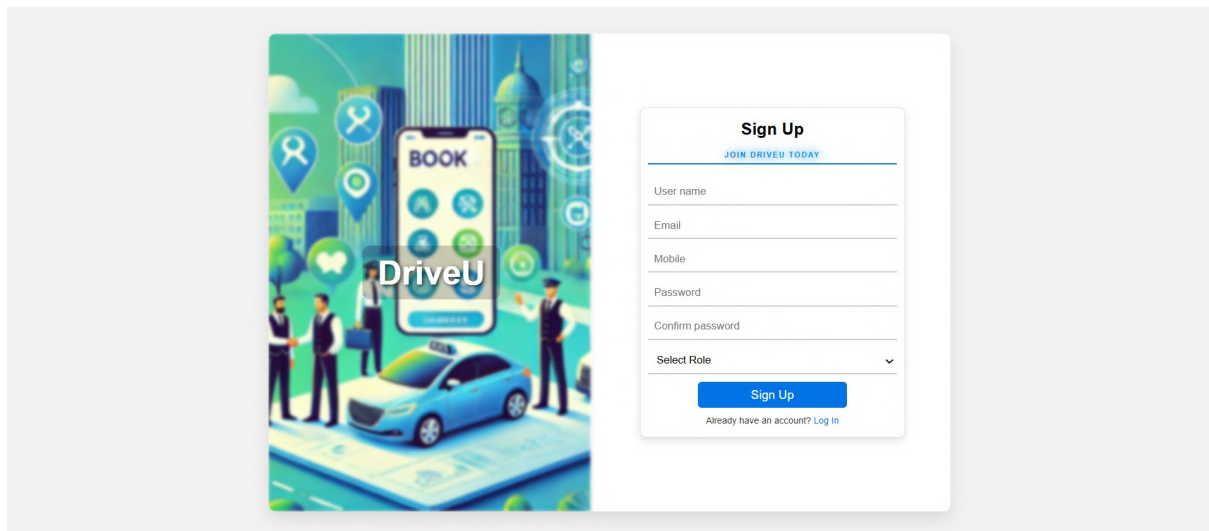
Component features a heading "DriveU" with background image (background.webp provided in images folder under assets)

Common Screens (Admin and Customer)

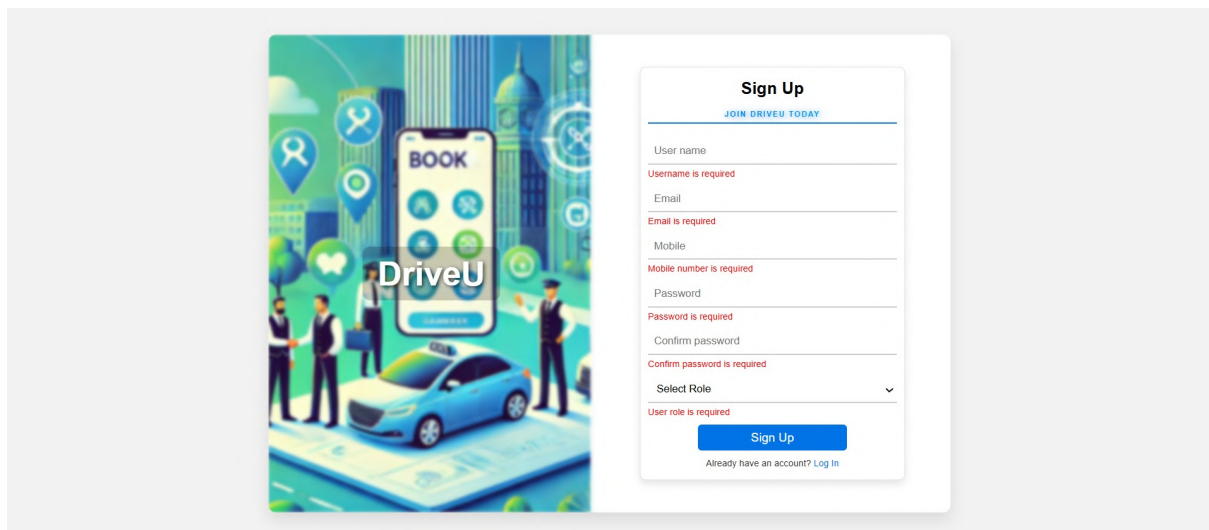
Signup Page:
(signup component)



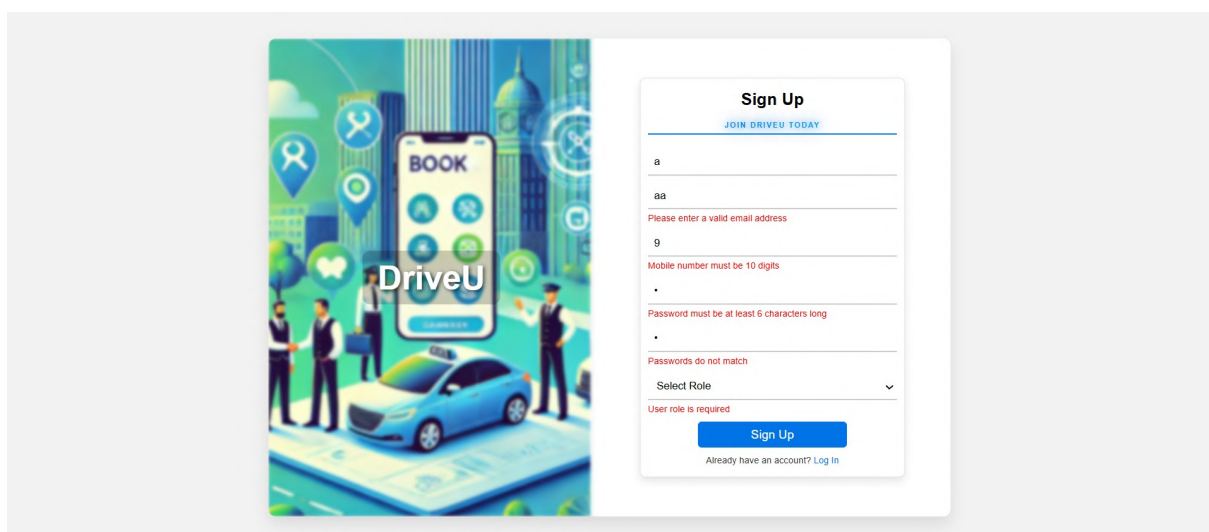
Clicking "**Sign Up**" in the login page will navigate to the registration page for both roles. Please refer to the screenshots below for validations.

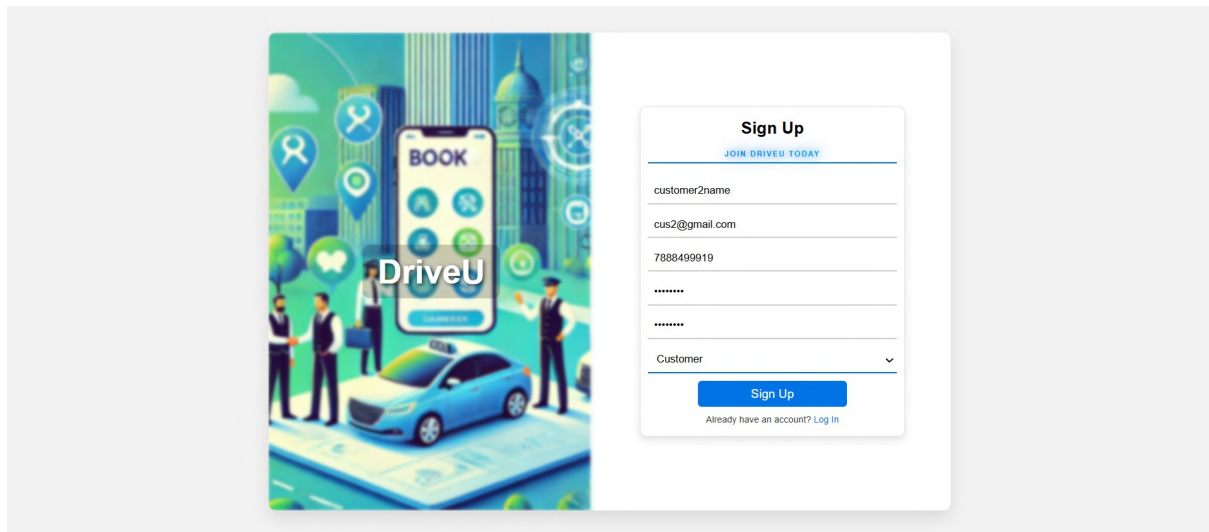


On clicking “Sign Up” button with empty field will display the validation message.

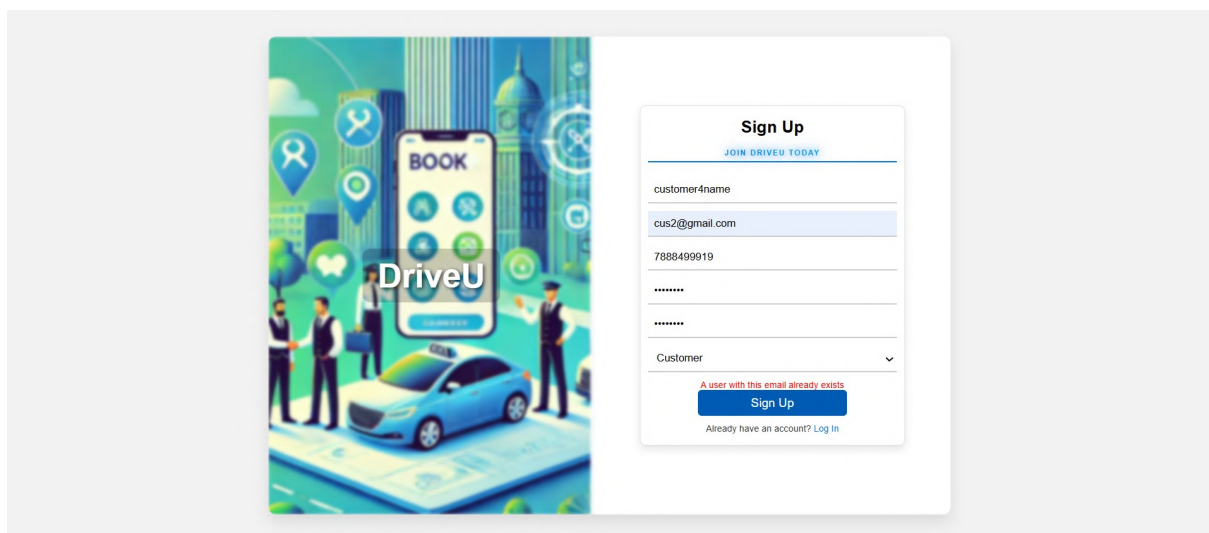


Regular expressions (Regex) are also included to validate email formats, mobile numbers, and to ensure password matching.

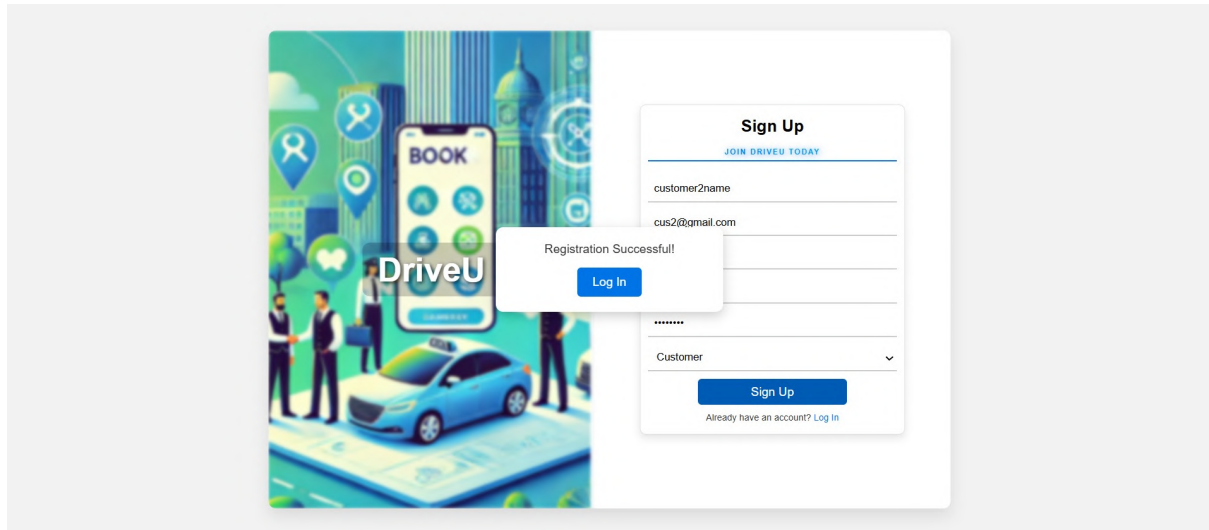




The message "A user with this email already exists" will be displayed if a signup attempt is made using an existing email.

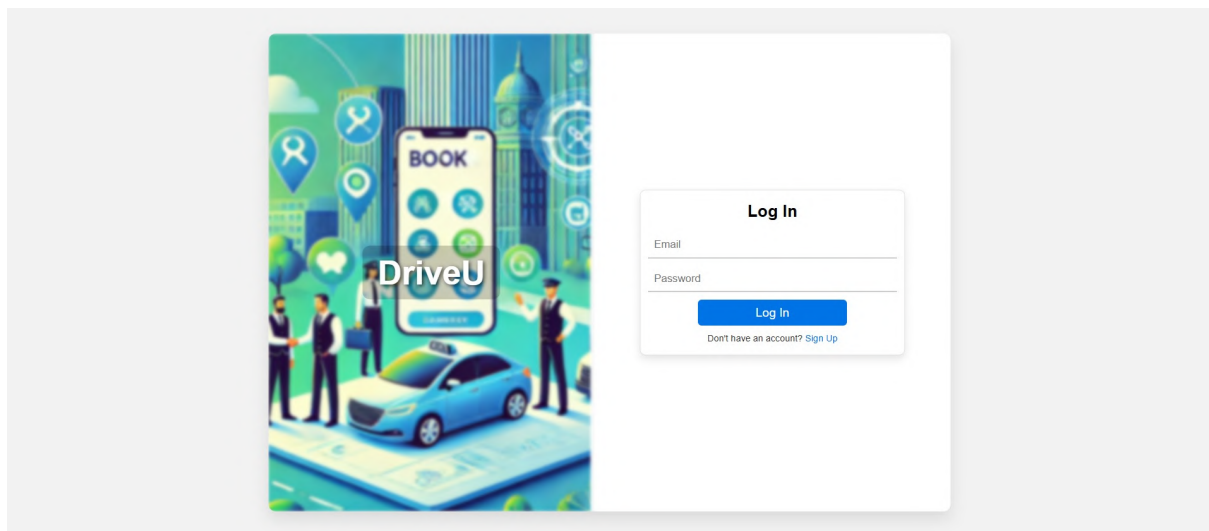


When the "Sign Up" button is clicked for valid data, upon successful completion, a modal will appear, prompting users to click the "Log In" button. Upon clicking this button, users will be directed to the login page, signifying a successful signup process.



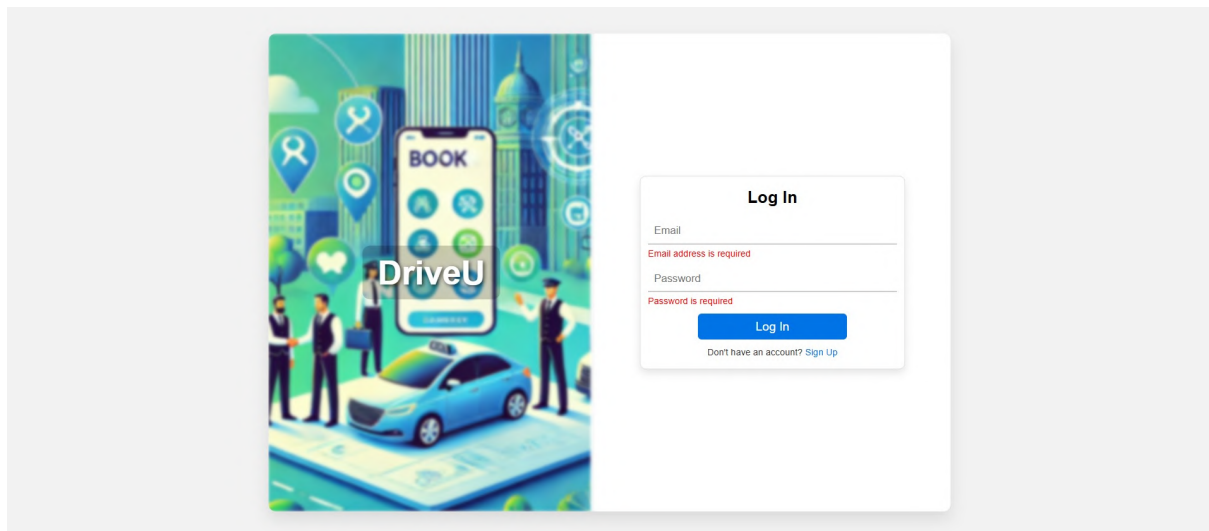
Login Page (login component)

This page is used for logging in to the application. On providing the valid email and password, the user will be logged in.

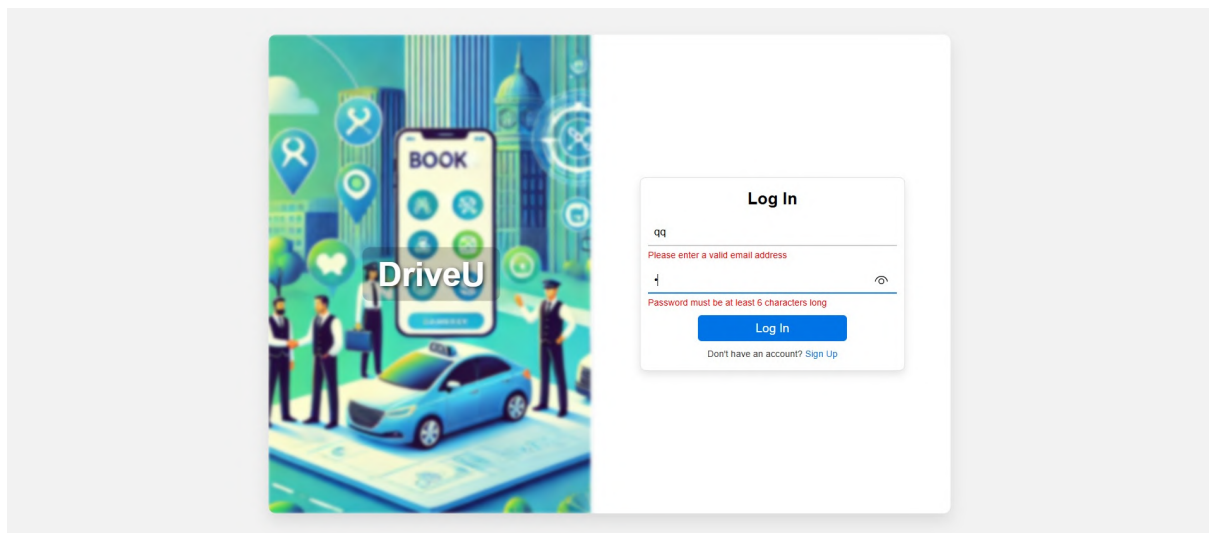


Perform validations for email and password fields.

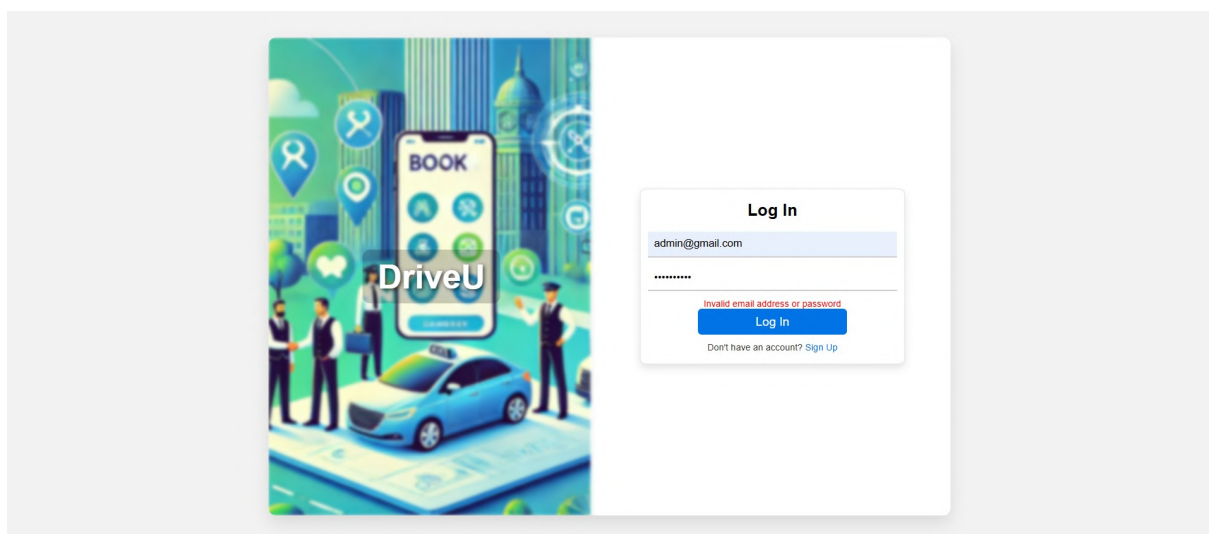
On clicking "Login" button with empty field will display the validation message.

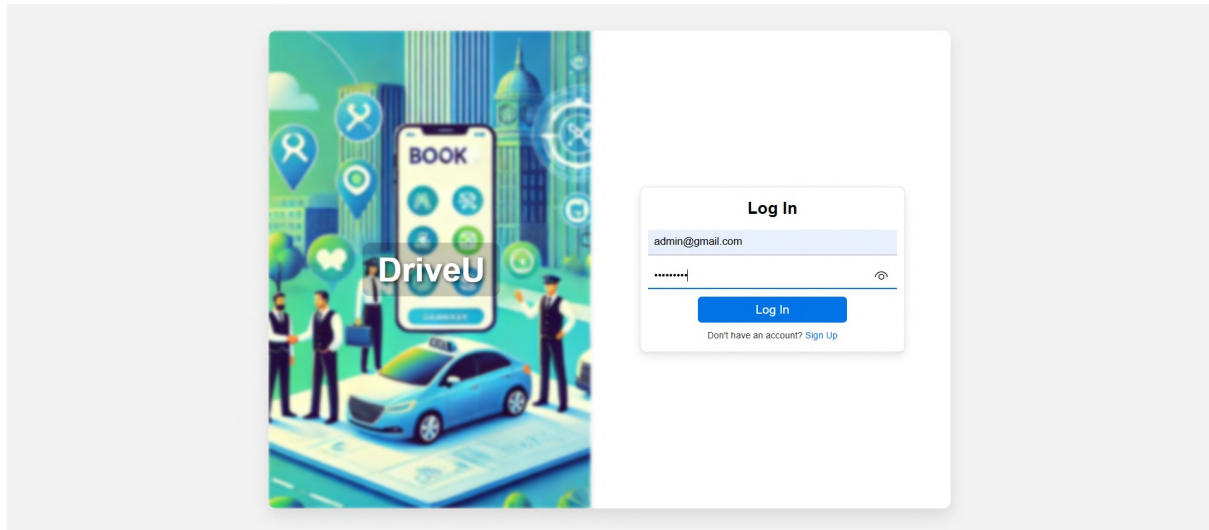


Regex validation:



Display the error message for Invalid Email or Password:



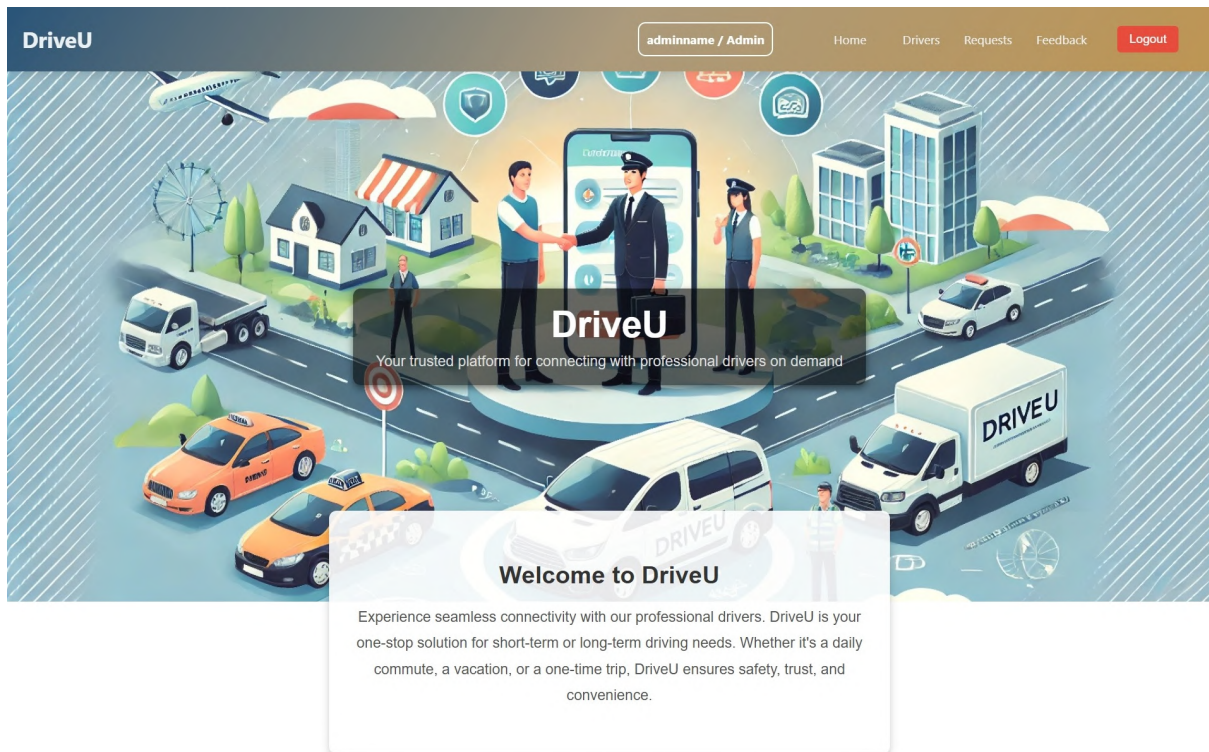


On Clicking the “Login” button, user will be navigated to the (adminnav or customernav component) based on their roles.

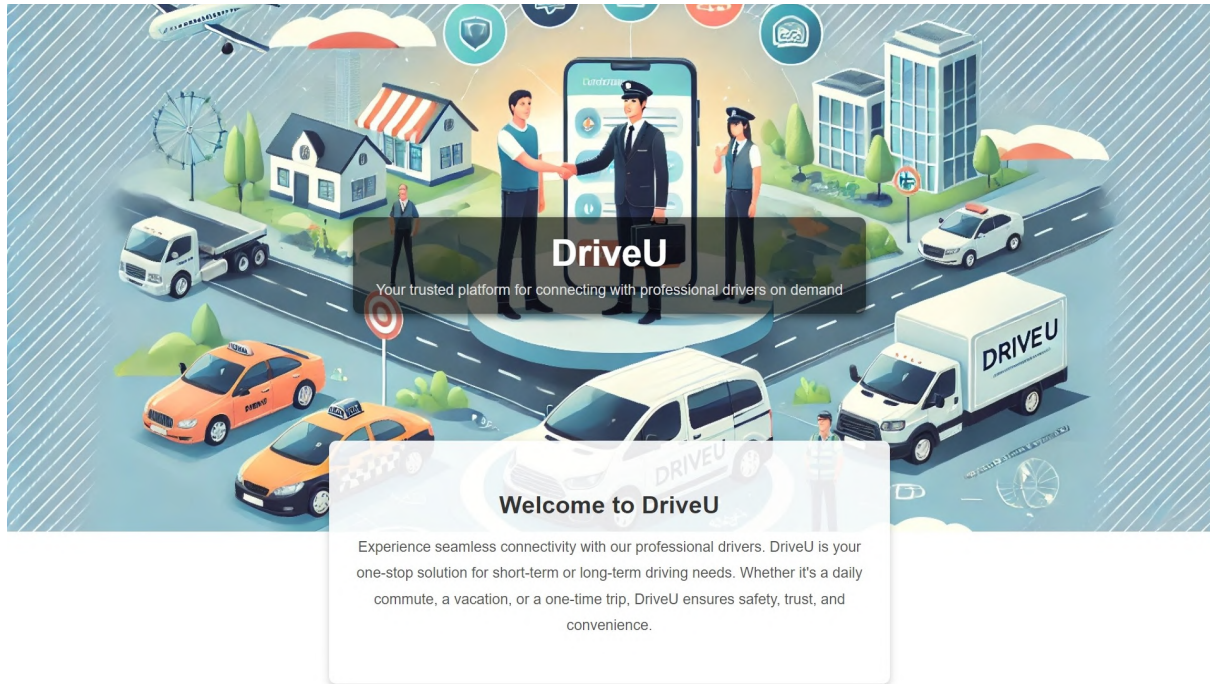
Admin side:

(adminnav component)

Upon successful login, if the user is an Customer, the (customernav component) will be displayed. If the user is Admin, the (adminnav component) will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.

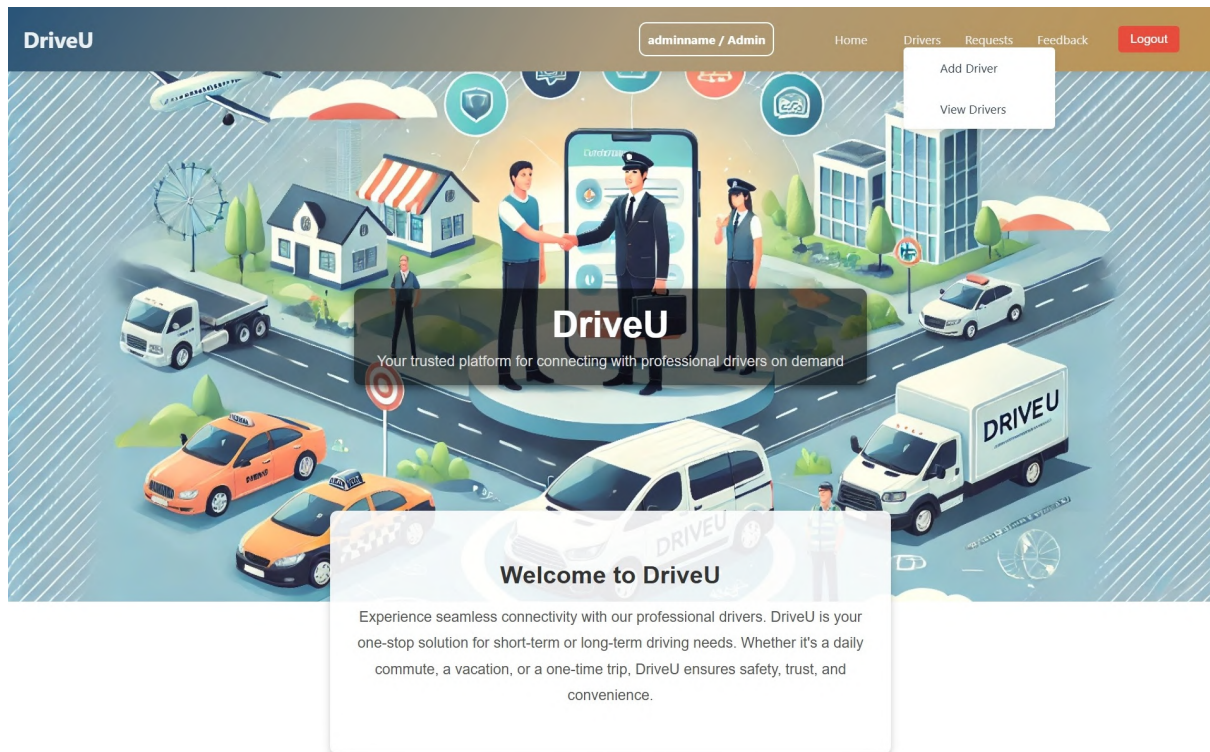


homePage Component: This page is used to display the information about DriveU application. On clicking the 'Home' tab, user can view the information about the application.(homepage.webp provided in images folder under assets)



Admins can navigate to other pages by clicking on the menu available in the navigation bar.

On hovering over the "Drivers" item in the navbar, a submenu should appear with options to "Add Driver" and "View Drivers".



Clicking on "Add Driver" will navigate to the **driver-management component**, which displays a form with the heading "**Create New Driver**".

Add New Driver

Name*

License Number*

Experience Years*

Contact Number*

Vehicle Type*

Choose Vehicle Type

Hourly Rate*

Address*

Driver Image*

[Choose File](#) No file chosen[Add Driver](#)

Perform validations for all the form fields.

Add New Driver

Name*

*Name is required

License Number*

*License Number is required

Experience Years*

*Experience is required

Contact Number*

*Contact Number is required

Vehicle Type*

Choose Vehicle Type

*Vehicle Type is required

Hourly Rate*

*Hourly Rate is required

Address*

*Address is required

Driver Image*

Choose File

No file chosen

Add Driver

Clicking the "Add Driver" button with empty fields will display a validation message stating "All fields are required".

Add New Driver

Name*

License Number*

Experience Years*

Contact Number*

Vehicle Type*

Choose Vehicle Type

Hourly Rate*

Address*

Driver Image*

 No file chosen

All fields are required

Add New Driver

Name*

demo name

License Number*

1234567890987654

Experience Years*

3

Contact Number*

91234567899

Vehicle Type*

SUV

Hourly Rate*

450

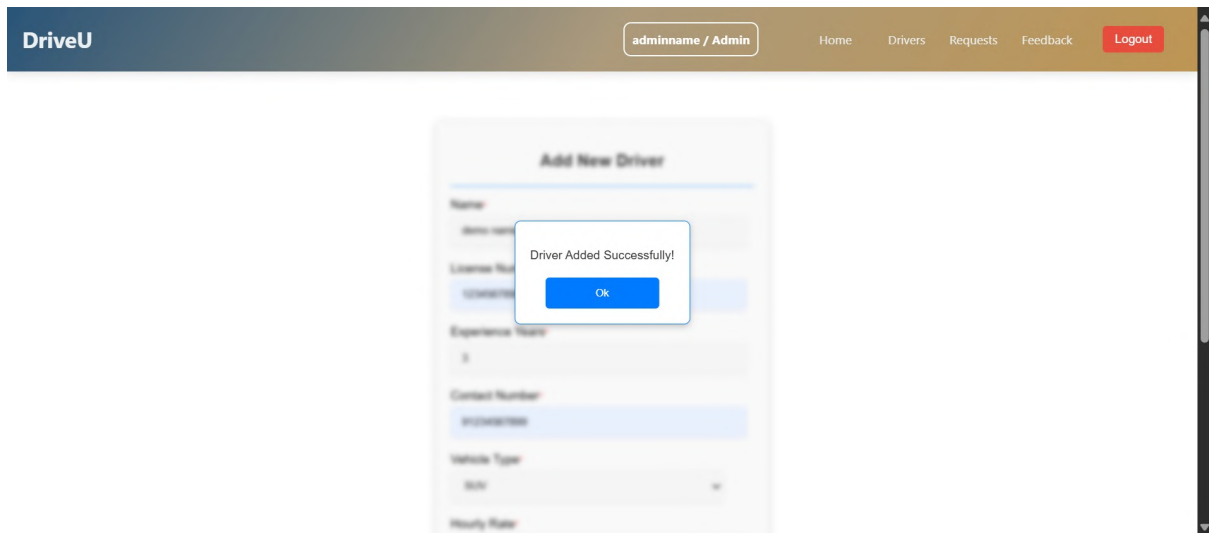
Address*

demo address

Driver Image*

 OIP (5).jpg

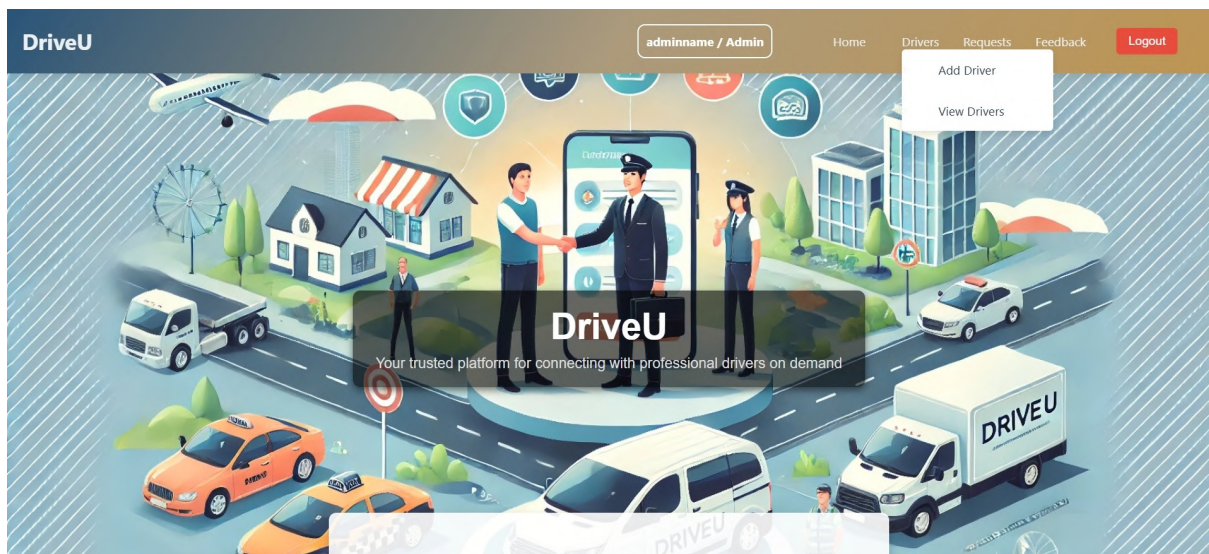
Upon clicking the "Add Driver" button, if the operation is successful, a popup message saying, "Driver Added Successfully!" should be displayed.



Upon clicking the "Ok" button, a driver is added to the system, and the form is reset, allowing the admin to add another driver. To navigate to other pages, the admin can use any menu option available in the navbar.

Admin View Drivers:

On hovering over the "Drivers" item in the navbar, a submenu should appear with options to "Add Driver" and "View Drivers".



Clicking on "View Drivers" will navigate to the **admin-view-drivers component**, which displays all driver details in a card format with the heading " Drivers". Additionally, a search feature is provided to search based on Driver Name and filter based on status.

If no data is available, then "Oops! No data found" should be displayed.

Drivers

Search by: Filter by Status: All Status 

Oops! No data found.

Drivers

Search by: Filter by Status: All Status 

demo name one

Vehicle Type: SUV
License: 1234567890987654
Phone: 91234567899
Address: demo address
Experience: 3 years
Hourly Rate: ₹450
Status: Inactive

[Edit](#)[Delete](#)[Action](#)

demo name two

Vehicle Type: Sedan
License: 1234567890987655
Phone: 91234567893
Address: demo address
Experience: 2 years
Hourly Rate: ₹230
Status: Active

[Edit](#)[Delete](#)[Action](#)

demo name three

Vehicle Type: Truck
License: 2234567890987651
Phone: 91234567894
Address: demo address
Experience: 7 years
Status: On Leave

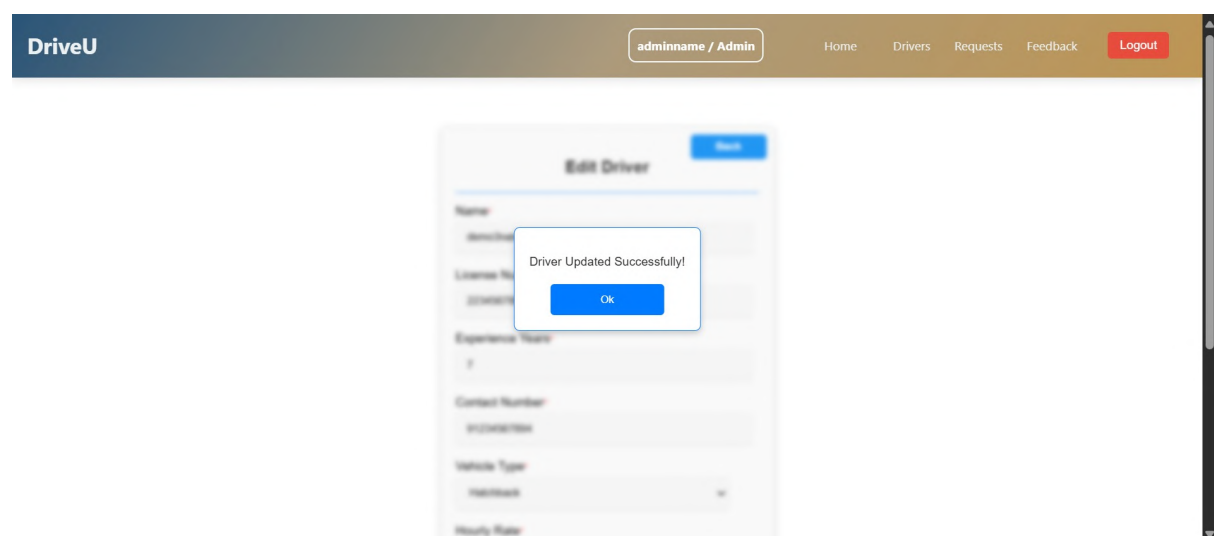
[Edit](#)[Delete](#)[Action](#)

Clicking the edit button will navigate to the **driver-management component**, which displays an editing form with pre-populated data of the selected driver. This form also contains a "Back" button that will navigate to the **admin-view-drivers component**. Validations are performed for all the form fields.

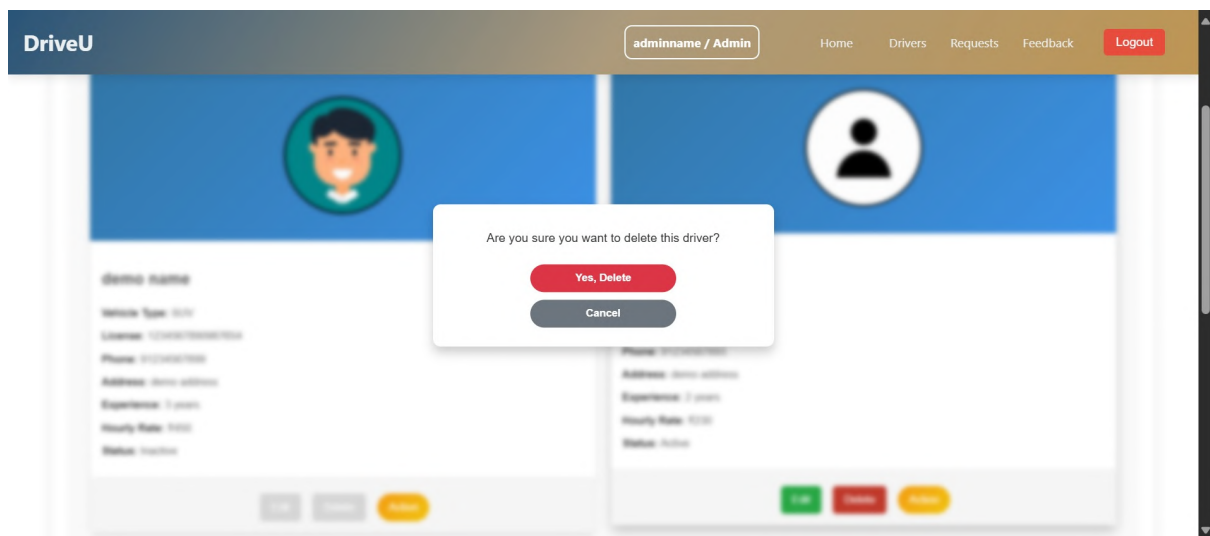
The Edit Driver form performs all validations in the same manner as the Add Driver form within the Driver Management component.

The screenshot shows the 'Edit Driver' form within the DriveU application. The header bar includes the 'DriveU' logo, a user profile 'adminname / Admin', and navigation links for 'Home', 'Drivers', 'Requests', 'Feedback', and a 'Logout' button. The form itself is titled 'Edit Driver' and features a 'Back' button in the top right corner. It contains several input fields: 'Name' (pre-filled with 'demo name three'), 'License Number' (pre-filled with '2234567890987651'), 'Experience Years' (pre-filled with '7'), 'Contact Number' (pre-filled with '91234567894'), 'Vehicle Type' (a dropdown menu currently showing 'Truck'), 'Hourly Rate' (pre-filled with '330'), 'Address' (pre-filled with 'demo address'), and 'Driver Image' (a file upload area with a 'Choose File' button and the text 'No file chosen'). At the bottom of the form is an 'Update Driver' button.

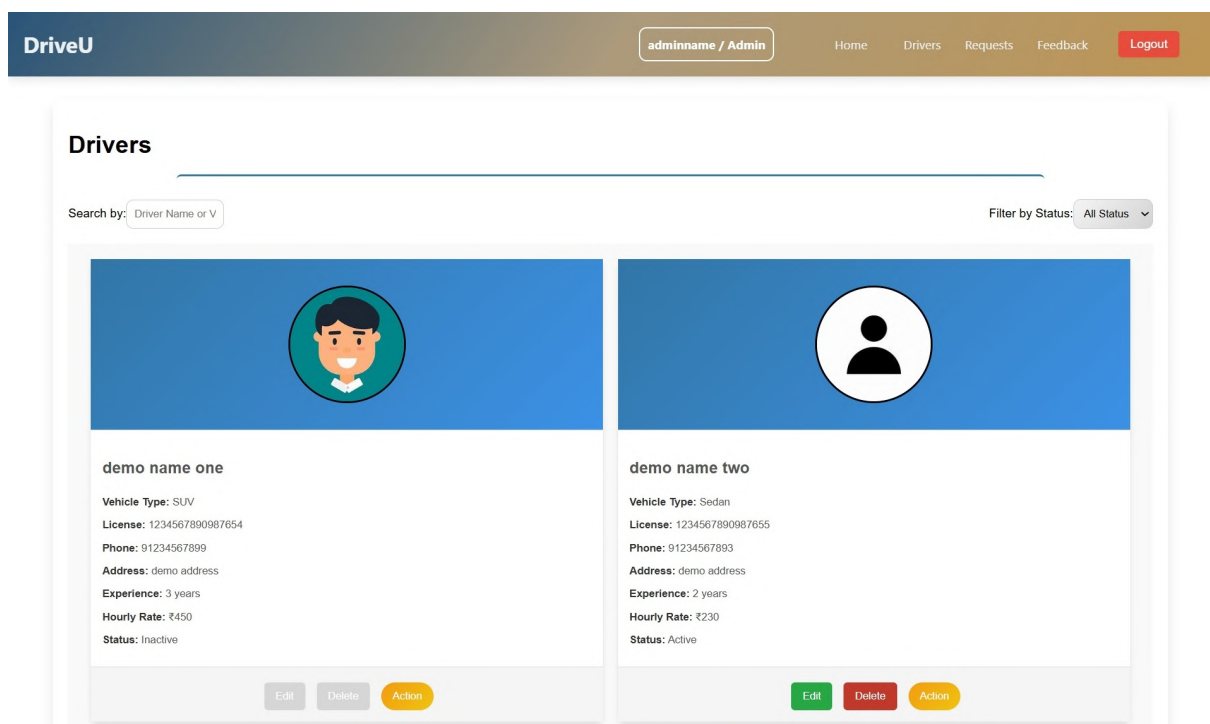
Upon clicking the "Update Driver" button, if the operation is successful, a popup saying "Driver Updated Successfully!" will be displayed. Clicking "OK" will redirect to the **admin-view- drivers component**.



On clicking the "Delete" button, a pop-up should be displayed with confirmatory message to delete the data.

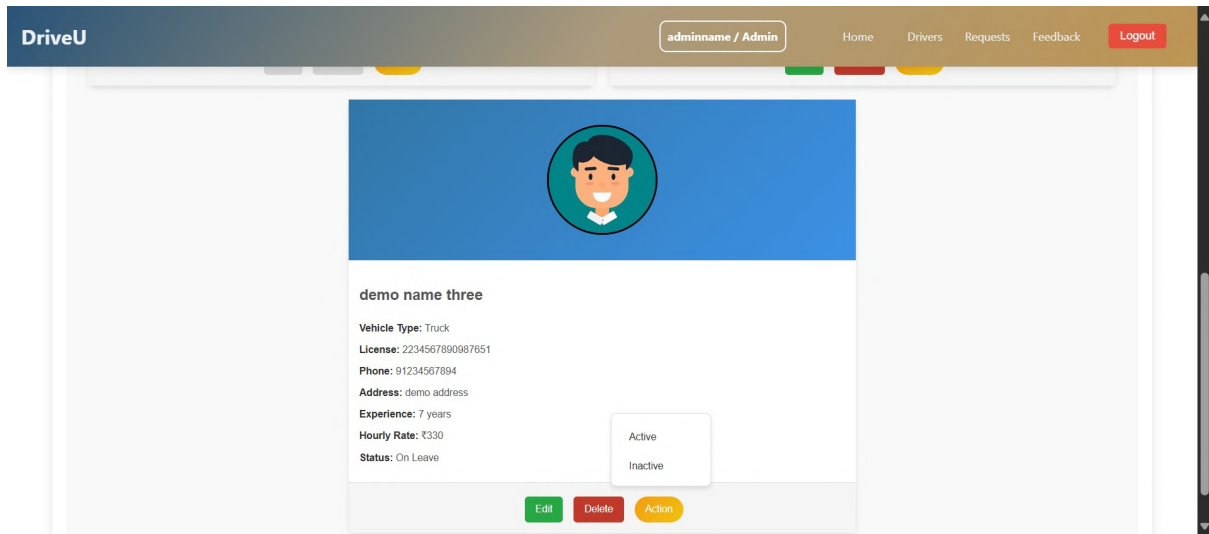


Clicking "Yes, Delete" button will delete the record. Clicking "Cancel" button will close the popup.

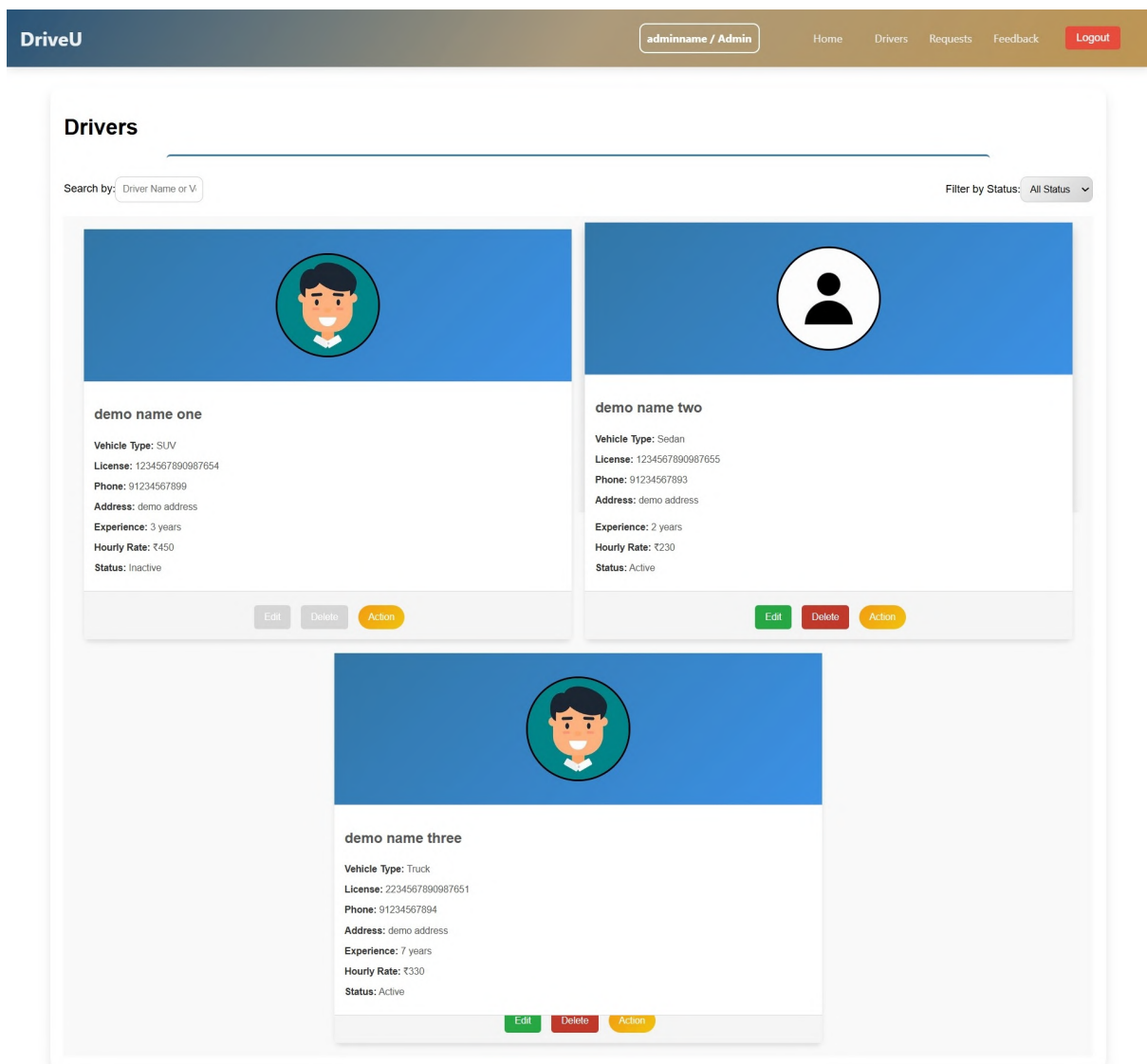


After successful deletion, the card should be refreshed as mentioned in the above images.

On clicking the "Action" button, a dropdown appears displaying status options (Active, On Leave, Inactive) excluding the driver's current status.



Selecting an option triggers an update to the driver's availability status and closes the dropdown.

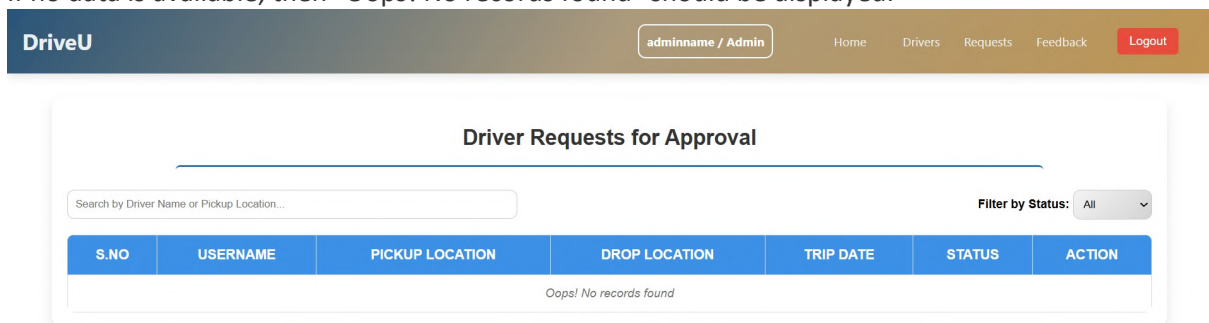


Admin View Requested

Clicking on "Requests" from the navbar will hover "View Requests" on clicking will navigate to the **adminviewrequests** component, which displays all driver requested by all customers.



If no data is available, then "Oops! No records found" should be displayed.



If data is available, the drivers requested by all customer will be displayed in table format. Additionally, features such as the admin being able to search based on Driver Name and filter the data based on Status.

DriveU

adminname / Admin

Home

Drivers

Requests

Feedback

Logout

Driver Requests for Approval

Search by Driver Name or Pickup Location...

Filter by Status: All

S.NO	USERNAME	PICKUP LOCATION	DROP LOCATION	TRIP DATE	STATUS	ACTION
1	demo	demo pick 2	demo drop 2	2025-01-25	Trip End	Show Driver Details View Stage
2	demo	demo pick 3	demo drop 3	2025-01-26	Pending	Show Driver Details Approve Reject View Stage
3	demo	demo pick 4	demo drop 4	2025-01-25	Trip End	Show Driver Details View Stage

The admin can approve or reject customer requests by clicking on the "Approve" or "Reject" button, respectively.

Action buttons such as "Show Driver Details," "Approve," and "Reject" are conditionally displayed to manage requests. These buttons enable users to perform context-specific actions seamlessly based on the request's current state.

DriveU

adminname / Admin

Home

Drivers

Requests

Feedback

Logout

Driver Requests for Approval

Search by Driver Name or Pickup Location...

Filter by Status: All

S.NO	USERNAME	PICKUP LOCATION	DROP LOCATION	TRIP DATE	STATUS	ACTION
1	demo	demo pick 2	demo drop 2	2025-01-25	Trip End	Show Driver Details View Stage
2	demo	demo pick 3	demo drop 3	2025-01-26	Approved	Show Driver Details Reject View Stage
3	demo	demo pick 4	demo drop 4	2025-01-25	Trip End	Show Driver Details View Stage

DriveU

adminname / Admin

Home

Drivers

Requests

Feedback

Logout

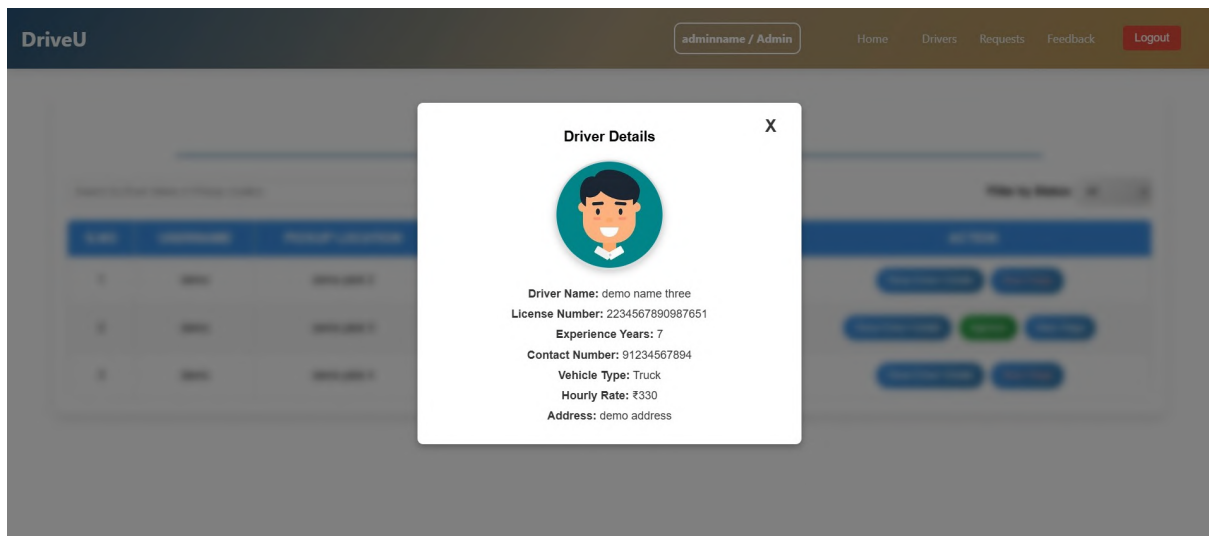
Driver Requests for Approval

Search by Driver Name or Pickup Location...

Filter by Status: All

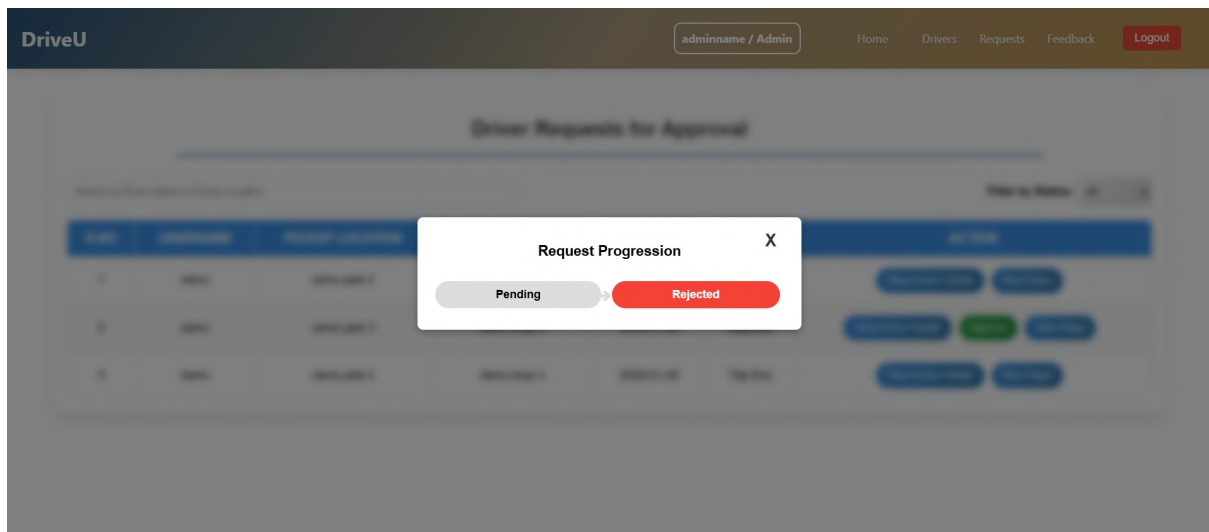
S.NO	USERNAME	PICKUP LOCATION	DROP LOCATION	TRIP DATE	STATUS	ACTION
1	demo	demo pick 2	demo drop 2	2025-01-25	Trip End	Show Driver Details View Stage
2	demo	demo pick 3	demo drop 3	2025-01-26	Rejected	Show Driver Details Approve View Stage
3	demo	demo pick 4	demo drop 4	2025-01-25	Trip End	Show Driver Details View Stage

Clicking on "Show Driver Details" will display additional details in a popup modal.

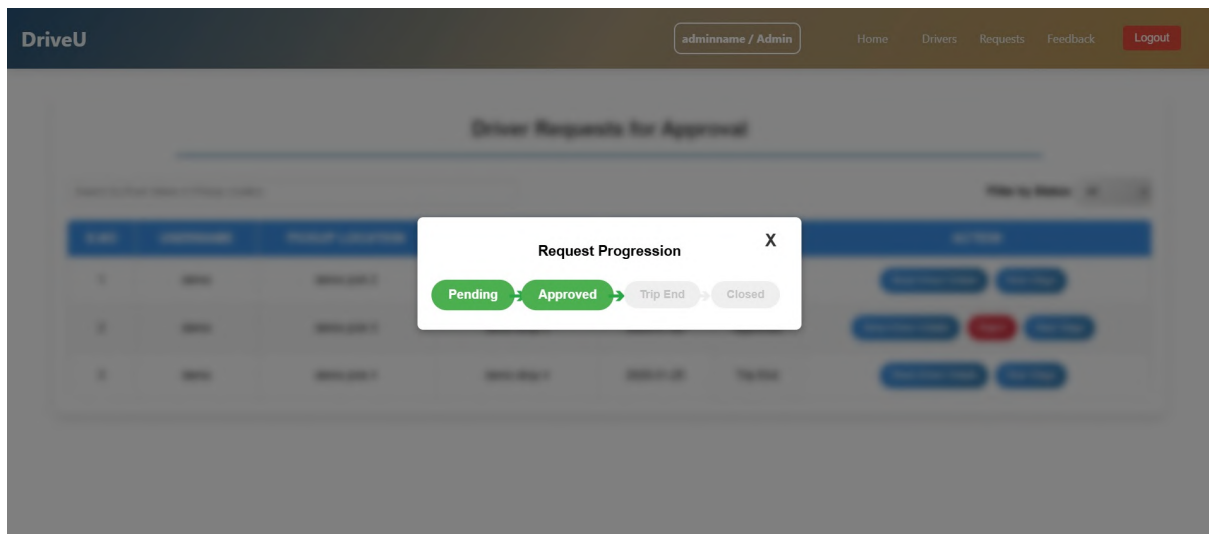


Each table row adapts its appearance dynamically based on specific conditions, including visual effects like blinking columns or buttons to highlight key actions or statuses.

The "View Stage" button opens a modal that visually represents the progression of a request. The modal provides an intuitive overview of different stages, making it easy to track progress or identify pending actions.

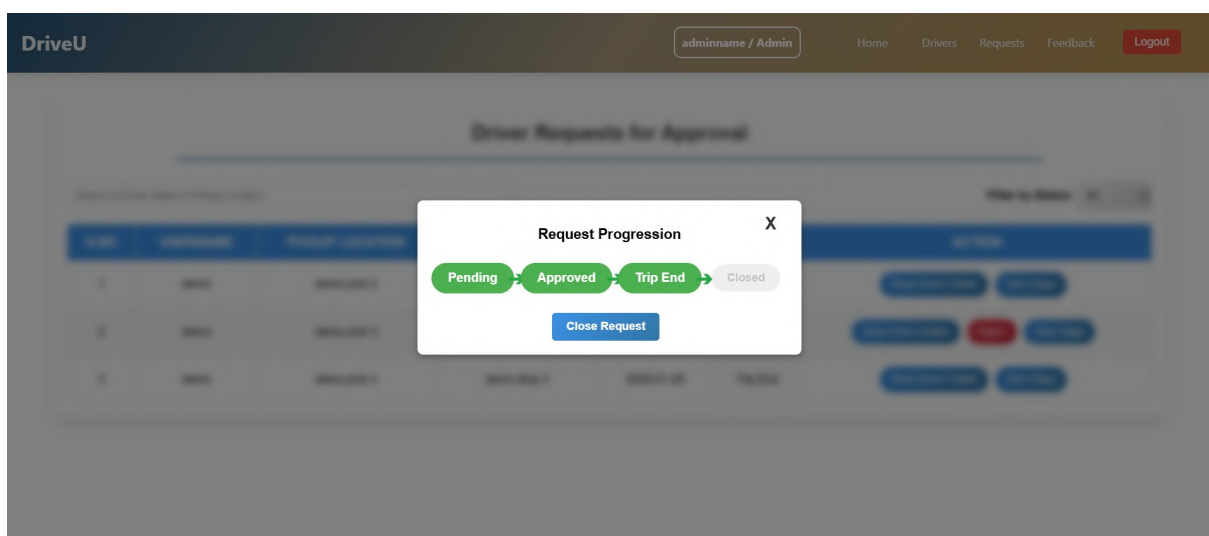


If a request is rejected by the admin, the progression in the modal visually updates to reflect the "Rejected" stage, clearly indicating the request's status in the workflow.



When a request is approved by the admin, the progression in the modal updates to display the "Approved" stage, with the next stage, "Trip End," updated by the customer upon completion of the trip.

A specific action button appears in the modal for requests that are ready for closure



A popup modal with a "Close Request" button will appear, which becomes enabled only when the stage is at "Trip End," allowing the request to be closed.

Clicking on the "X" sign will close the modal.

DriveU

adminname / Admin

[Home](#)
[Drivers](#)
[Requests](#)
[Feedback](#)
[Logout](#)

Driver Requests for Approval

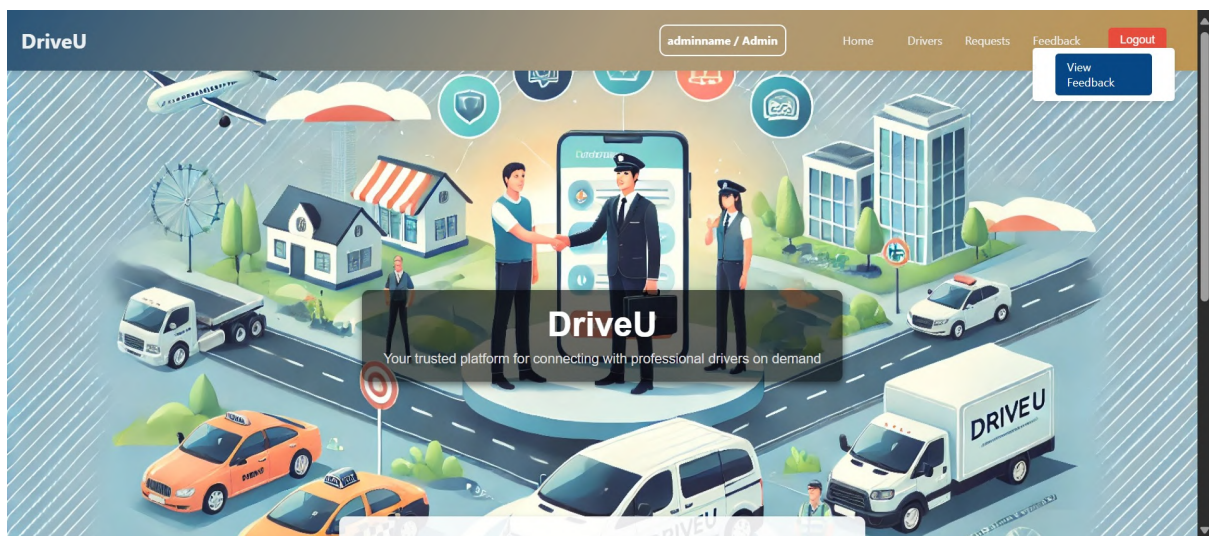
Search by Driver Name or Pickup Location...

Filter by Status: All

S.NO	USERNAME	PICKUP LOCATION	DROP LOCATION	TRIP DATE	STATUS	ACTION
1	demo	demo pick 2	demo drop 2	2025-01-25	Closed	Show Driver Details View Stage
2	demo	demo pick 3	demo drop 3	2025-01-26	Approved	Show Driver Details Reject View Stage
3	demo	demo pick 4	demo drop 4	2025-01-25	Trip End	Show Driver Details View Stage

Admin View Feedbacks:

Clicking on “Feedback” from the navbar will hover “View Feedback” on clicking will navigate to the **adminviewfeedback component**, which displays all feedbacks posted by all customers.



If no data is available, then "No data found" should be displayed.

DriveU

adminname / Admin

Home

Drivers

Requests

Feedback

Logout

Feedback Received

Filter by Category: All Categories

S.NO	USER NAME	FEEDBACK	POSTED DATE	ACTION
No Data Found				

If data is available, the feedbacks posted by all customers will be displayed in table format. Admin can filter based on category.

DriveU

adminname / Admin

Home

Drivers

Requests

Feedback

Logout

Feedback Received

Filter by Category: All Categories

S.NO	USER NAME	FEEDBACK	POSTED DATE	ACTION
1	demo	excellent in punctuality	25/01/2025	Show Profile View Driver Info
2	demo	demo performance	25/01/2025	Show Profile View Driver Info

Clicking on "Show Profile" will display additional details about the user in pop-up modal.

DriveU

adminname / Admin

Home

Drivers

Requests

Feedback

Logout

Feedback Received

Filter by Category: All Categories

S.NO	USER NAME	FEEDBACK	POSTED DATE	ACTION
1	demo	excellent in punctuality	25/01/2025	Show Profile View Driver Info
2	demo	demo performance	25/01/2025	Show Profile View Driver Info

User Details:

Email: cus@gmail.com

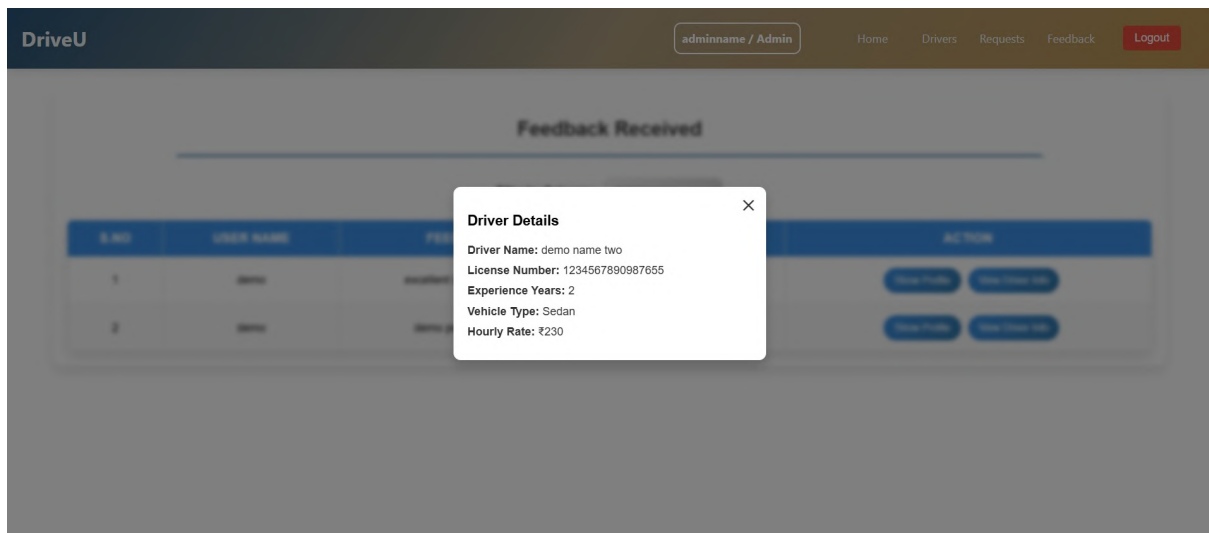
Username: demo

Mobile Number: 9123456789

Close

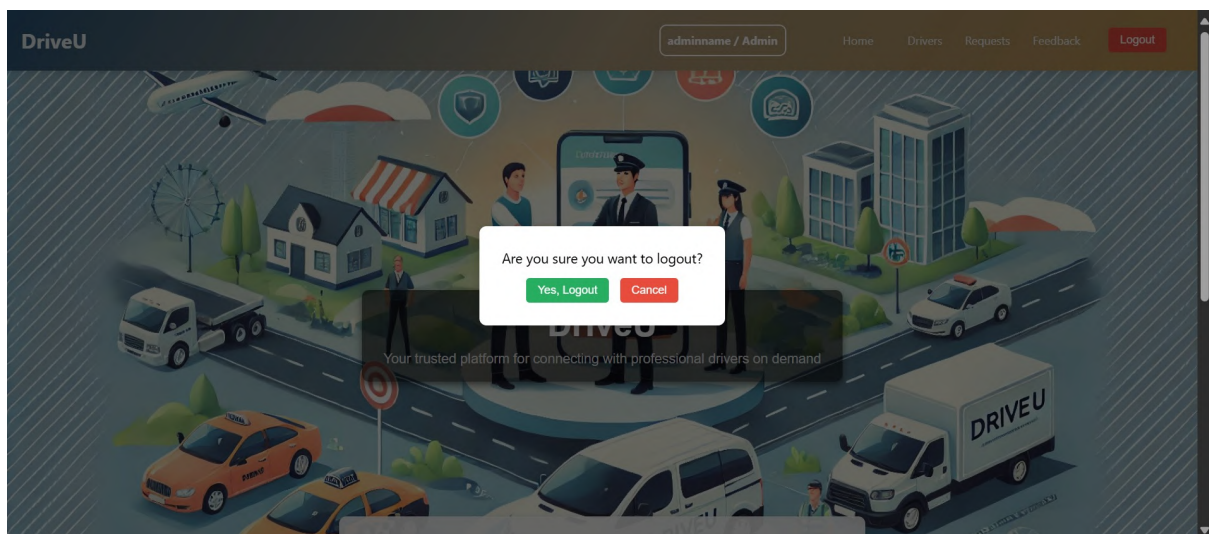
On clicking the "Close" button, will close the pop-up modal displayed.

Clicking on "View Driver Info" will display additional details about the user in pop-up modal.

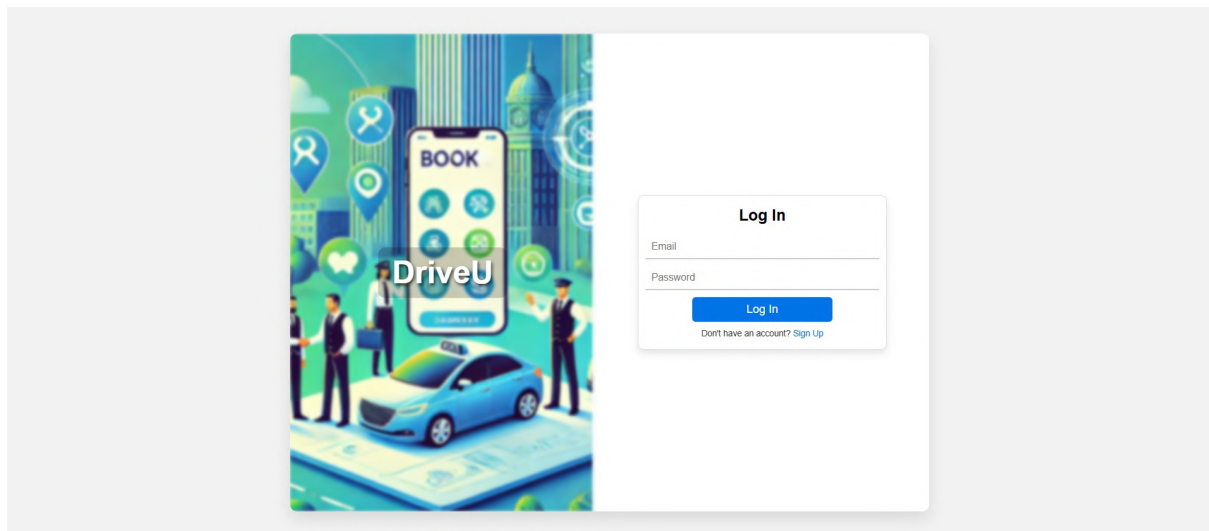


On clicking the "Close" button, will close the pop-up modal displayed.

On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.

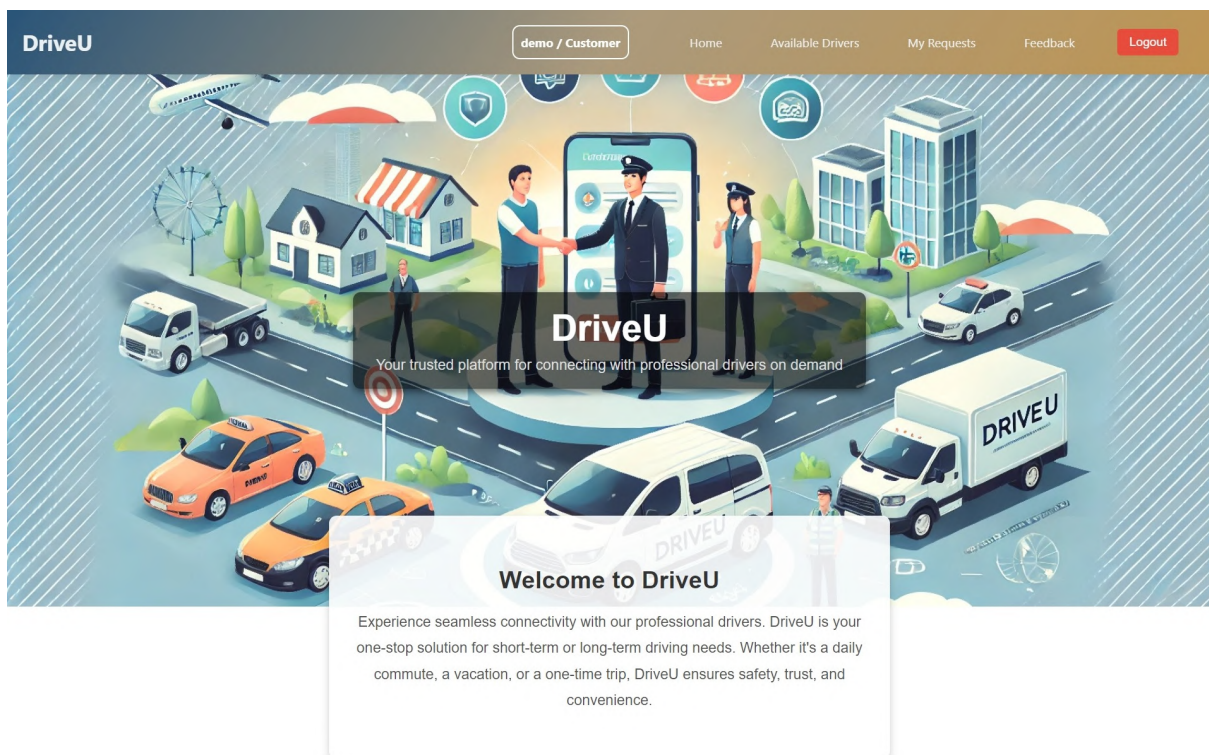


Clicking "Yes, Logout" will navigate to the login component.



Customer side:

Upon successful login, if the user is an admin, the **(adminnav component)** will be displayed. If the user is a regular Customer, the **(customernav component)** will be displayed. Additionally, the role-based navigation bar will also display login information such as the username and role.

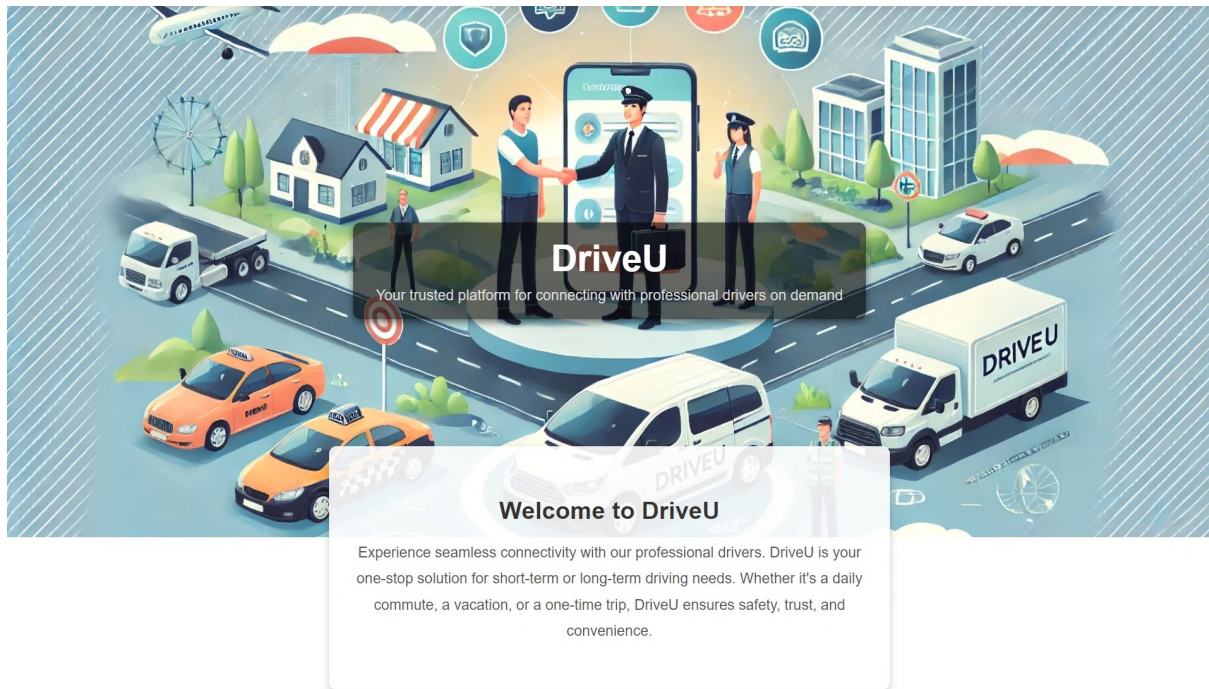


Contact Us

Phone: 987-654-3210

Email: support@driveu.com

homePage Component: This page is used to display the information about DriveU application. On clicking the 'Home' tab, user can view the information about the application.(homepage.webp provided in images folder under assets)



Contact Us

Phone: 987-654-3210

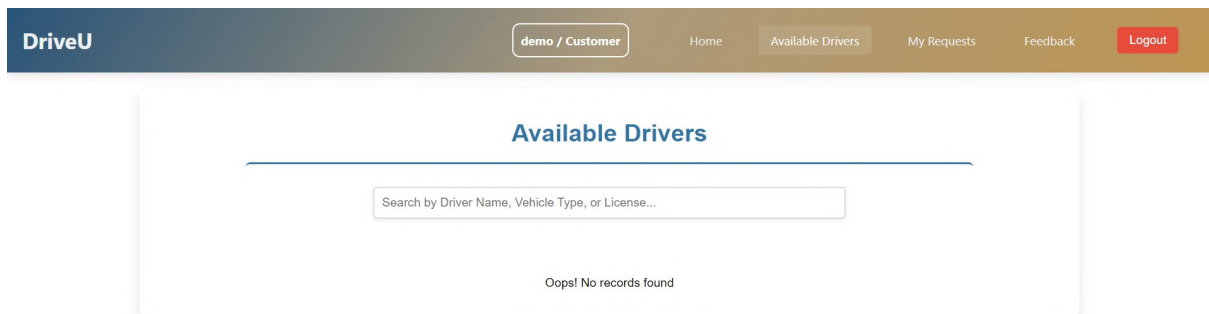
Email: support@driveu.com

User View Drivers:

Clicking "Available Drivers" in the navigation bar navigates to the **customerviewdriver** component, displaying a list of available drivers posted by admins.



If no data is available, then "Oops! No records found" should be displayed.



A list of available drivers is rendered dynamically as individual cards. Each card displays the driver's image, name, vehicle type, license number, contact information, address, years of experience, hourly rate, and current availability status.

DriveU

demo / Customer

Home

Available Drivers

My Requests

Feedback

Logout

Available Drivers

Search by Driver Name, Vehicle Type, or License...



demo name one

Vehicle Type: SUV

License: 1234567890987654

Phone: 91234567899

Address: demo address

Experience: 3 years

Hourly Rate: ₹450

Status: Inactive

Request



demo name two

Vehicle Type: Sedan

License: 1234567890987655

Phone: 91234567893

Address: demo address

Experience: 2 years

Hourly Rate: ₹230

Status: Active

Request



demo name three

Vehicle Type: Truck

License: 2234567890987651

Phone: 91234567894

Address: demo address

Experience: 7 years

Hourly Rate: ₹330

Status: Active

Request

Each driver card includes a "Request" button, which is enabled only if the driver is active and has not already been requested. If the driver has been requested, a "Requested" button replaces the "Request" button for clarity.

If Status is Inactive or On Leave, then Request button will be disabled.

Clicking the "Request" button will navigate to the **customer-request component**, which displays a form with the heading **"New Driver Request"**. Additionally, it includes a "Back" button, which, when clicked, moves back to the **customerviewdriver component**

DriveU

demo / Customer

Home

Available Drivers

My Requests

Feedback

Logout

New Driver Request

Back

Pickup Location*

Drop Location*

Trip Date*

dd-mm-yyyy

Time Slot*

--:--

Estimated Duration*

Comments

Submit Request

Perform validations for all the form fields.

New Driver Request

Back

Pickup Location*

*Pickup Location is required

Drop Location*

*Drop Location is required

Trip Date*

dd-mm-yyyy

*Trip Date is required

Time Slot*

--:--

*Time Slot is required

Estimated Duration*

*Estimated Duration is required

Comments

Submit Request

Clicking the "Submit Request" button with empty fields will display a validation message stating " All required fields must be filled out".

New Driver Request

Back

Pickup Location*

Drop Location*

Trip Date*

dd-mm-yyyy

Time Slot*

--:--

Estimated Duration*

Comments

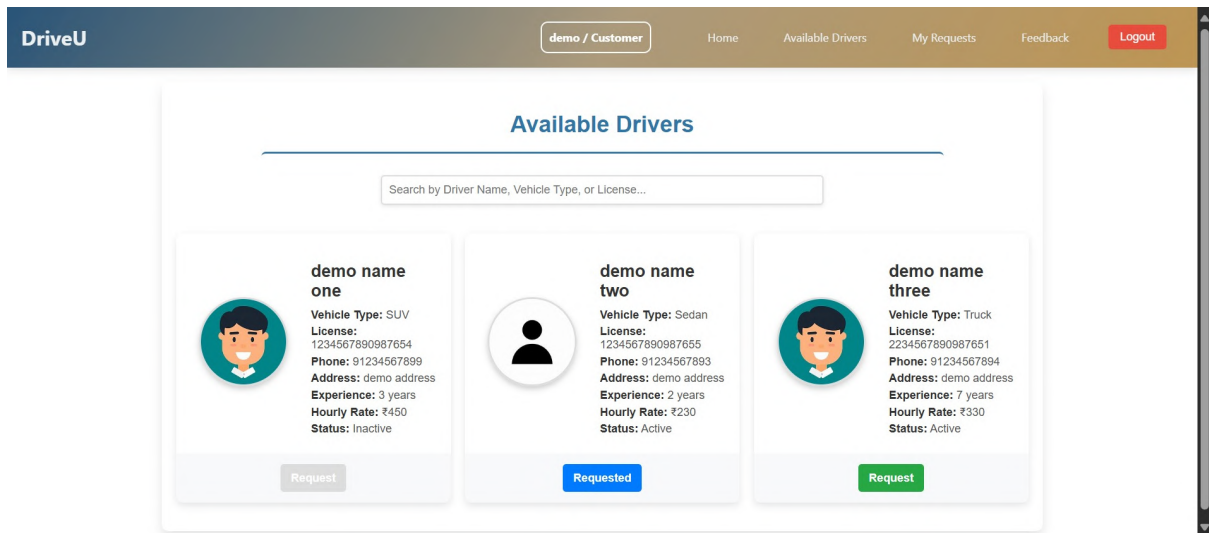
All required fields must be filled out.

Submit Request

Upon clicking the "Submit Request" button with valid data, if the operation is successful, a popup message saying "Request Submitted Successfully!" should be displayed.

The image shows two screenshots of the DriveU web application. The top screenshot displays the 'New Driver Request' form. The form includes fields for 'Pickup Location' (demo pick 2), 'Drop Location' (demo drop 2), 'Trip Date' (06-01-2025), 'Time Slot' (22:56), 'Estimated Duration' (3 hours), and a 'Comments' section (demo comments). A 'Submit Request' button is at the bottom. The bottom screenshot shows the same form after a successful submission, with a modal popup displaying 'Request Submitted Successfully!' and an 'OK' button. The background form is blurred. Both screenshots feature a top navigation bar with the DriveU logo, a 'demo / Customer' dropdown, and links for Home, Available Drivers, My Requests, Feedback, and a Logout button.

Clicking "Ok" will redirect to the **customerviewdriver component**. If the customer has requested for the driver, it will display as "Requested"; otherwise, it will display "Request".



User Applied Requests:

Clicking "My Requests" in the navigation bar navigates to the **customerviewrequested** component, which displays the customer applied requests.



If no data is available, then "No Data Found" should be displayed.

Requested Drivers

No Data Found

If data is available, the drivers requested will be displayed in card format. Additionally, features such as the user being able to search based on driver Name.

Requested Drivers

demo driver one
Vehicle Type: SUV
Experience: 7 years
Pickup: demo pick 2
Drop: demo drop 2
Status: Trip End

Show MoreEditDeleteTrip EndFetch PayAmount

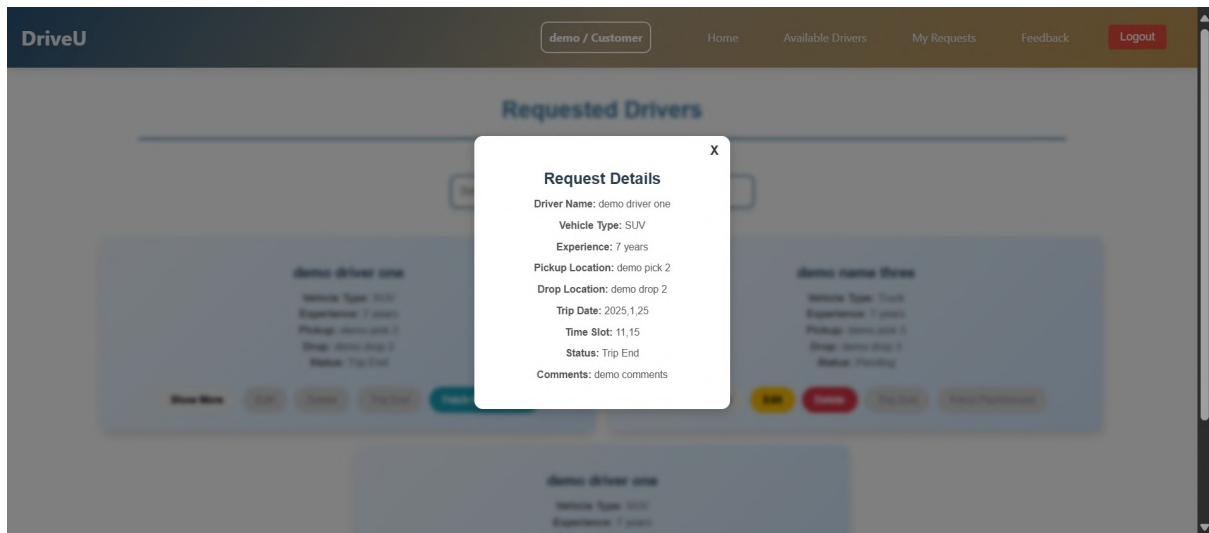
demo name three
Vehicle Type: Truck
Experience: 7 years
Pickup: demo pick 3
Drop: demo drop 3
Status: Pending

Show MoreEditDeleteTrip EndFetch PayAmount

demo driver one
Vehicle Type: SUV
Experience: 7 years
Pickup: demo pick 4
Drop: demo drop 4
Status: Pending

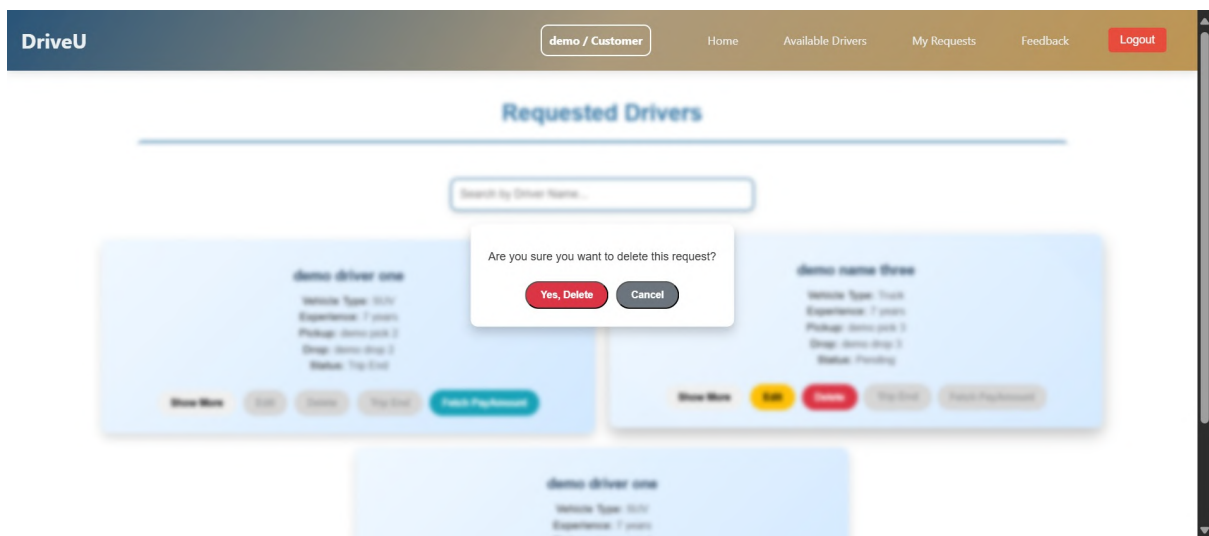
Show MoreEditDeleteTrip EndFetch PayAmount

Clicking on “Show More” will display additional details about the request in pop-up modal.



On clicking “X” will close the popup – modal displayed.

On clicking the "Delete" button, a pop-up should be displayed with confirmatory message to delete the data.



Clicking "Yes, Delete" will delete the applied request, and the change will be automatically reflected.

Clicking “Cancel” will close the popup modal.

On clicking the "Edit" button, users navigate to a page to **customer-request component** with prepopulated data of selected request ,where they can update the request details. Additionally, with back button.

Back

Edit Request

Pickup Location*
demo pick 4

Drop Location*
demo drop 4

Trip Date*
25-01-2025

Time Slot*
14:25

Estimated Duration*
2 hours

Comments
demo comments

Update Request

Validations are performed for all the form fields.

The Edit Request form performs all validations in the same manner as the Add Driver Request form within the Customer Request component.

Back

Edit Request

Pickup Location*
demo pick 4

Drop Location*
demo drop 4

Trip Date*
25-01-2025

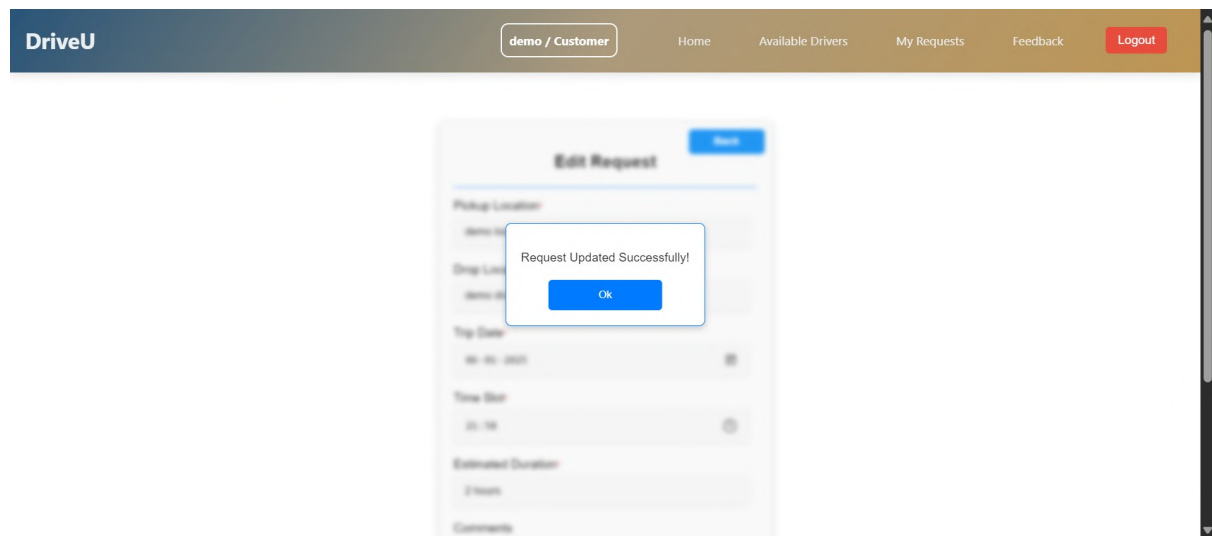
Time Slot*
14:25

Estimated Duration*
2 hours

Comments
demo comments updated

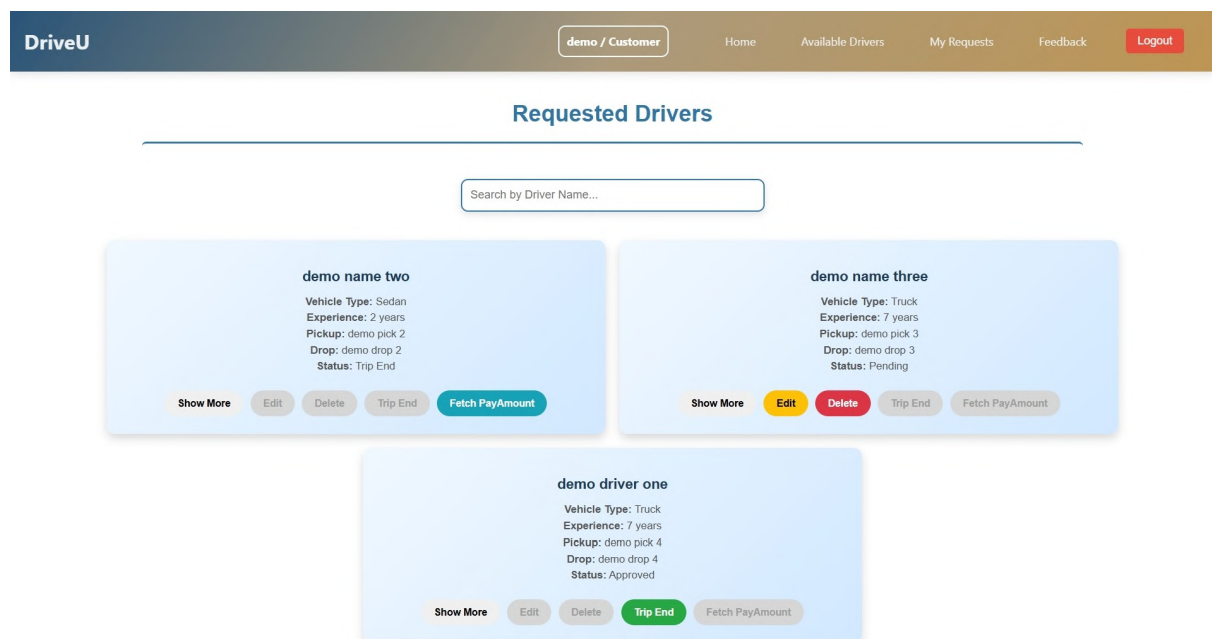
Update Request

Upon clicking the "Update Request" button, if the operation is successful, a popup saying "Request Updated Successfully!" will be displayed. Clicking "OK" will redirect to the **customer-request** component.



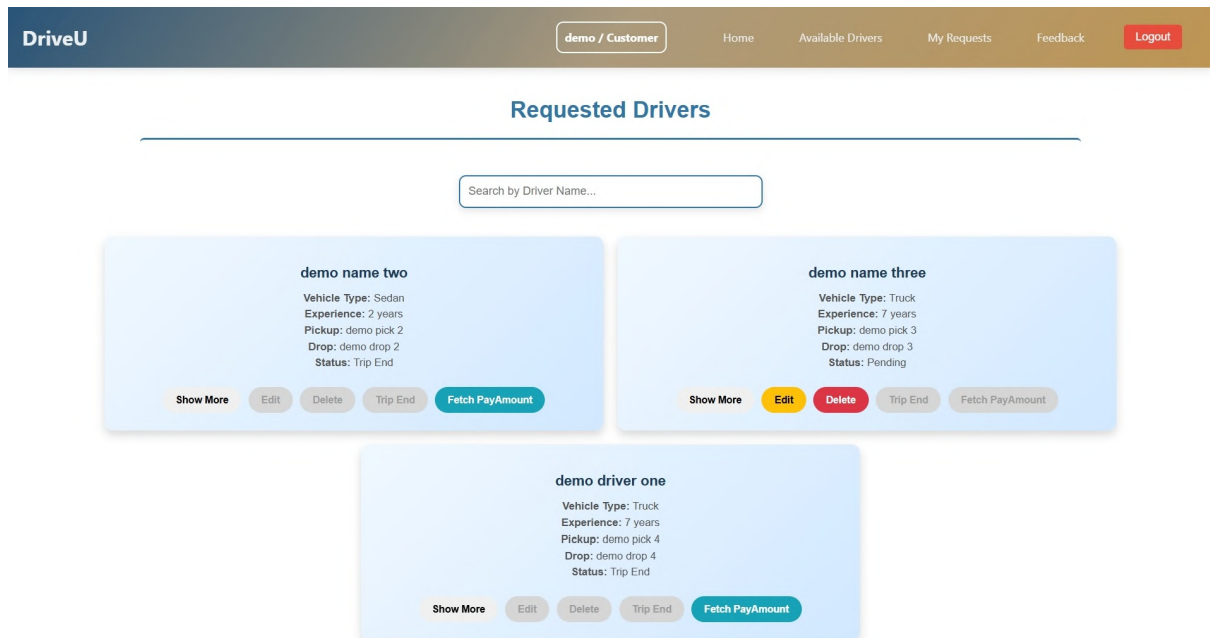
The **Edit** and **Delete** buttons are accessible only when the request status is in "Pending." These buttons are disabled for all other statuses, ensuring that modifications or deletions can only be performed on pending requests.

The **"Trip End"** button is accessible only when the request status is set to **Approved**, ensuring that trips can only be marked as Trip ends after they have been officially approved. For all other statuses, the button remains disabled.



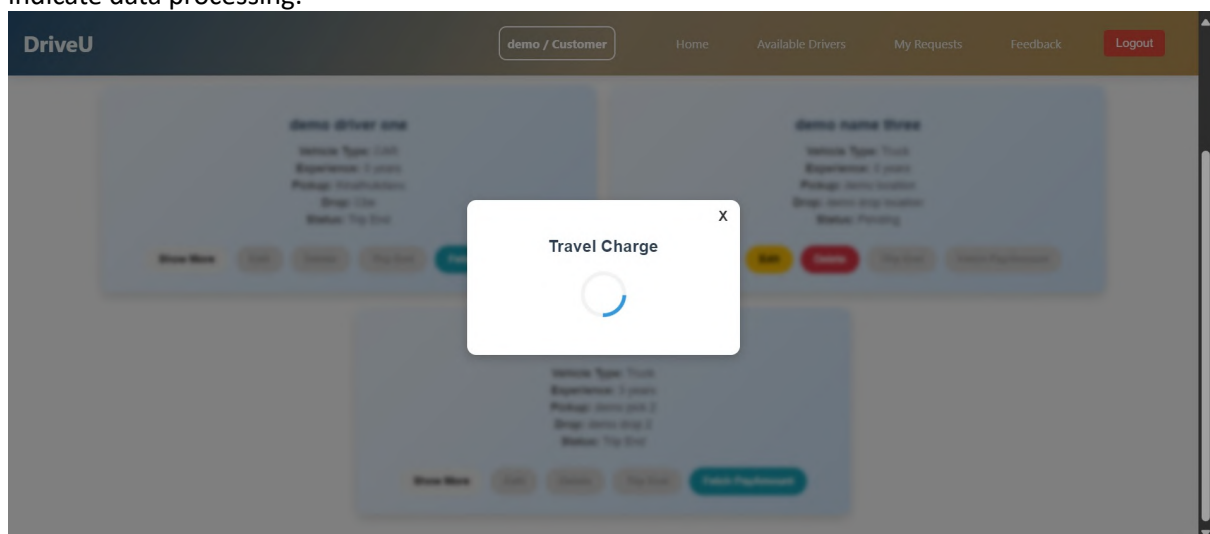
Clicking **"Trip End"** triggers the process to mark the trip as completed by capturing the current date and time. The system calculates the trip duration by comparing the trip's start time with the actual

end time and formats it in hours and minutes. Based on the duration, the payment amount is computed using the driver's hourly rate. The updated details, including the actual drop time, duration, payment, and status, are prepared and sent to the backend for updating the request. Once the update is successful, the UI refreshes to reflect the changes. This action enables further steps like fetching payment details or providing feedback.



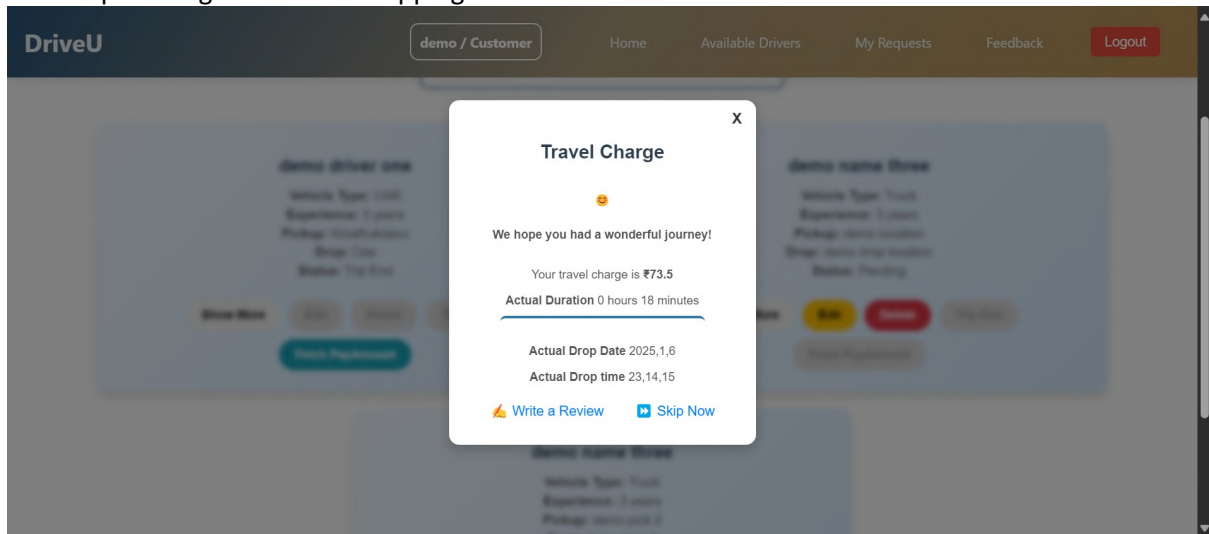
Once the **"Trip End"** button is clicked, the trip status is updated, and the payment details, including the trip duration and amount, are calculated. After this update, the **"Fetch PayAmount"** button becomes accessible, allowing the user to view the payment details and proceed with further actions like reviewing

On clicking **"Fetch Pay Amount"**, a popup modal is displayed with an initial loading spinner to indicate data processing.



After a brief delay, the modal updates to show the calculated travel charge, trip duration, and drop details, allowing the user to review the payment information and proceed with additional actions,

such as providing feedback or skipping the review.



Clicking on “Write a Review” will navigate to the **customerpostfeedback component**, which displays the form to post feedback with heading as "Submit Feedback"

Clicking “X” or Skip Now will close the popup modal.

A screenshot of the 'Submit Feedback' form. The form is titled 'Submit Feedback' and is enclosed in a light gray box. It contains three input fields: 'Feedback *' (a large text area), 'Category *' (a dropdown menu with 'Choose Category' selected), and 'Rating (1-5) *' (a star rating input). A blue 'Submit' button is at the bottom of the form.

Clicking the "Submit" button with empty field will display a validation message.

Submit Feedback

Feedback *

*Feedback is required.

Category *

Choose Category

*Category is required.

Rating (1-5) *

*Rating must be between 1 and 5.

Submit

Submit Feedback

Feedback *

demo feedback

Category *

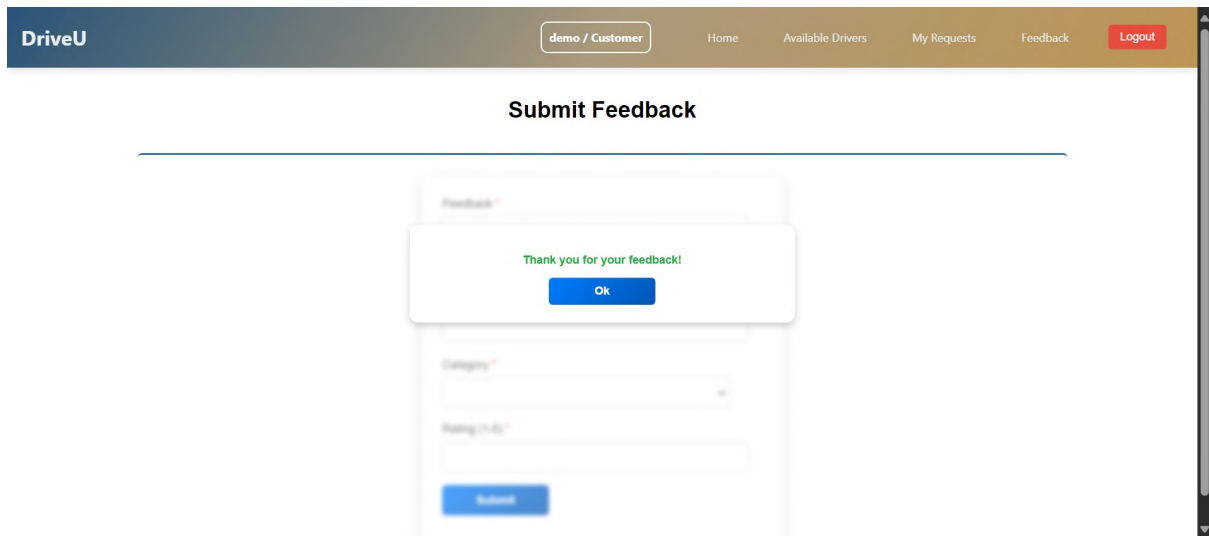
Service Experience

Rating (1-5) *

2

Submit

Upon clicking the "Submit" button, if the operation is successful, a popup message saying "Thank you for your feedback!" should be displayed.



User View Feedback:

Clicking "Feedback" in the navigation bar navigates to the **customerviewfeedback** component, which displays posted feedback with heading as "My Feedback"



If data is not available to display, then display's No Data Found

My Feedback

S.No	Feedback	Category	Rating	Action
No Feedback Found				

My Feedback

S.No	Feedback	Category	Rating	Action
1	excellent in punctuality	Punctuality	5	<button>View Driver Info</button> <button>Delete</button>
2	demo performance	Driver Performance	2	<button>View Driver Info</button> <button>Delete</button>
3	demo feedback	Service Experience	2	<button>View Driver Info</button> <button>Delete</button>

On clicking the "View Driver Info" button, a pop-up should be displayed with driver details.

DriveU

demo / Customer

Home

Available Drivers

My Requests

Feedback

Logout

My Feedback

S.No	Feedback	Category	Rating	Action
1	excellent in punctuality	Punctuality	5	<button>View Driver Info</button> <button>Delete</button>
2	demo performance	Driver Performance	2	<button>View Driver Info</button> <button>Delete</button>
3	demo feedback	Service Experience	2	<button>View Driver Info</button> <button>Delete</button>

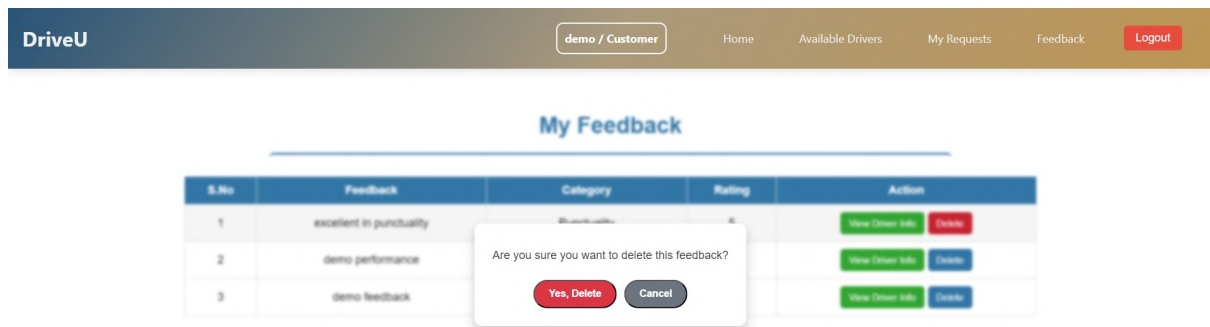
Driver Details

×

Driver Name: demo driver one
License Number: 1234567890987654
Experience: 3 years
Phone Number: 9876543214

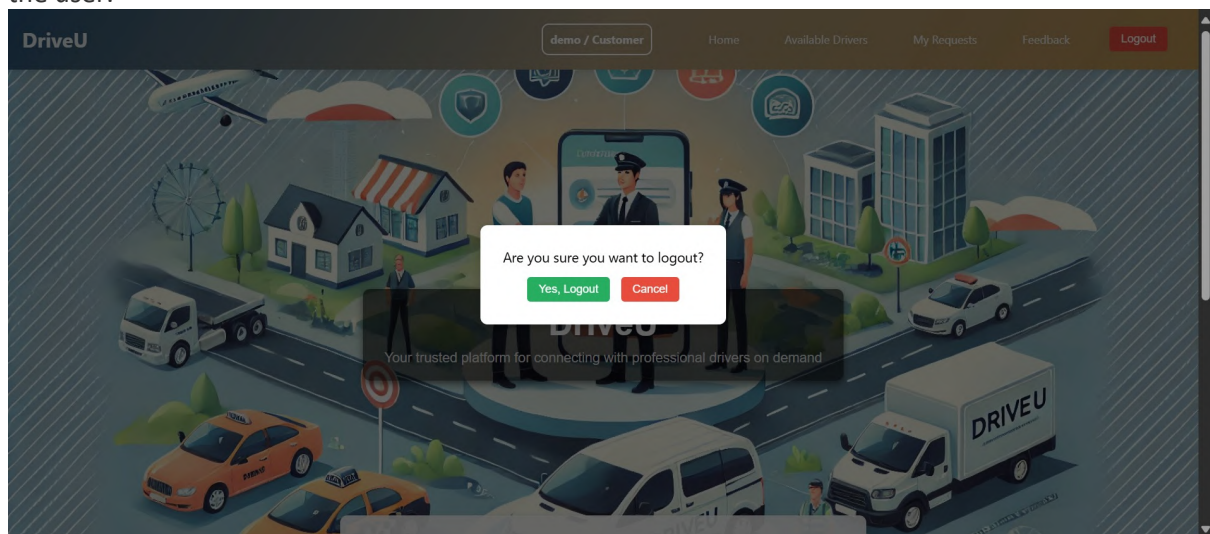
Clicking on "X" symbol, will close the modal.

On clicking the "Delete" button, a pop-up should be displayed with confirmatory message to delete the data.

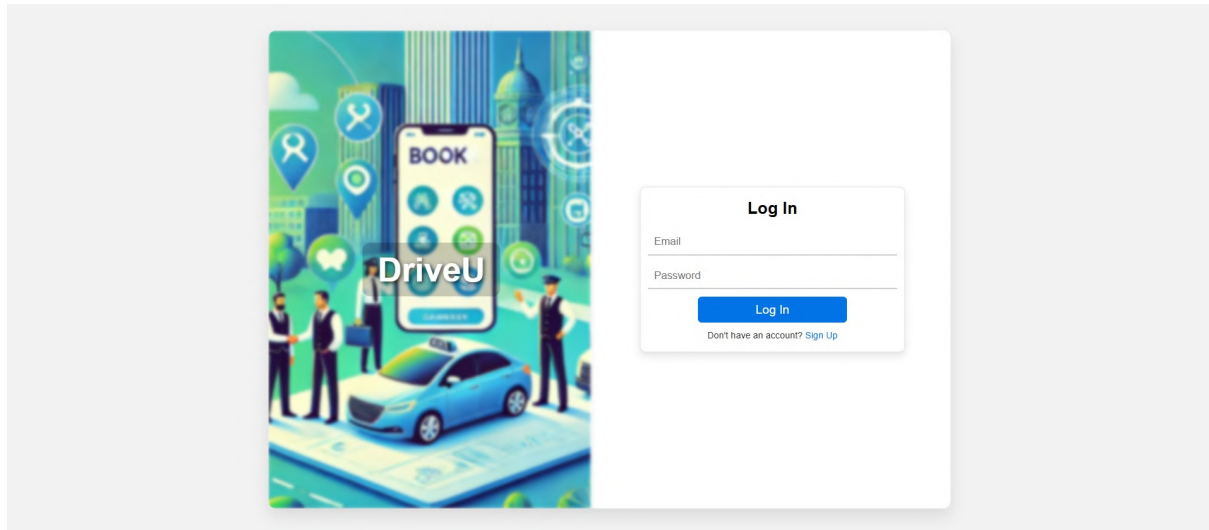


Clicking "Yes, Delete" will delete the feedback posted, and the change will be automatically reflected.

On clicking the "Logout" button, a pop-up should be displayed with confirmatory message to logout the user.



Clicking "Yes, Logout" will navigate to the login component.



error component

This page displays an error message stating "Something Went Wrong".

Something Went Wrong

We're sorry, but an error occurred. Please try again later.

Platform Prerequisites (Do's and Don'ts):

1. The angular app should run in port 8081.
2. The springapp should run in port 8080.
3. To incorporate Spring Security into the application, utilize either basic authentication or JWT authentication within the project workspace. For auto-evaluation purpose, before submitting the code, please verify that all security-related files, including the security dependency in the pom.xml, have been commented out. Alternatively, include GET: "/api/driver" and GET: "/api/feedback" APIs in permitAll() in the security config to allow all users to access the APIs

Key points to remember:

1. The id (for frontend) and attributes(backend) mentioned in the SRS should not be modified at any cost. Failing to do may fail test cases.
2. Remember to check the screenshots provided with the SRS.

3. Strictly adhere to the proper project scaffolding (Folder structure), coding conventions, method definitions and return types.
4. Adhere strictly to the endpoints mentioned in API endpoints section.
5. Don't delete any files in a project environment.

HOW TO RUN THE PROJECT:

BACKEND:

Open the terminal and follow the commands below.

- **cd springapp**

To run the application in port 8080

- **mvn spring-boot: run**

If the same error persists clean the project and build again

To Work with MySQLServer:

(Open a New Terminal) type the below commands

mysql -u root --protocol=tcp -p

password: **examly**

Note:

1. Please ensure that the application is running on port 8080 before clicking the "Run Test Case" button.
2. Database Name should be **appdb**

FRONTEND:

Open the terminal and follow the commands below.

Step 1:

- Use "**cd angularapp**" command to go inside the angularapp folder
- Install Node Modules - "**npm install**"

Step 2:

- Write the code inside src/app folder
- Create the necessary components
- To create Service: "**npm ng g s <service name>**"
- To create Component: "**npm ng g c <component name>**"

Step 3:

- Click the **Run Test Case** button to run the test cases

Note:

- Click PORT **8081** to view the result / output.
- If any error persists while running the app, delete the node modules and reinstall them.

AI Integrated Features (Phase 2)

As part of the LTM AI-integration initiative, DriveU has been enhanced with three AI-driven features powered by the Google Gemini API. These features reuse a single shared GeminiService (embeddings + text generation) and all degrade gracefully to a non-AI fallback when the API key is not configured, so existing functionality and test cases remain intact.

AI Features Overview:

1. **Semantic Driver Search (Embeddings + Vector Similarity):** Customers can search for drivers using natural language (e.g. "experienced SUV driver for a night outstation trip"). The query and each driver profile are converted to embeddings and ranked by cosine similarity.
2. **Feedback Sentiment Analysis & Auto-Tagging (LLM Classification):** When a customer submits feedback, Gemini classifies its sentiment (Positive / Neutral / Negative), produces a sentiment score, and extracts theme tags. Admins can view and filter feedback by sentiment.
3. **Per-Driver AI Review Summary (LLM Summarization):** For any driver, all feedback is aggregated by Gemini into a concise digest of strengths, areas to improve and an overall AI quality score.

AI Backend Endpoints:

AiController API Endpoints

Action	URL	Method	Role	Response
Semantic Driver Search	/api/ai/driver-search	POST	All	Returns response code 200 with a relevance-ranked List<Driver> for the natural-language query in the request body.
Driver Review Summary	/api/ai/driver/{driverId}/feedback-summary	GET	Admin, Customer	Returns response code 200 with an AI summary object (summary, strengths, improvements, aiScore, averageRating, feedbackCount).
Analyze Existing Feedback	/api/ai/feedback/analyze-existing	POST	Admin	Returns response code 200 with { updated: n } after back-filling sentiment and tags for feedback created earlier.

Updated Model Class - Feedback (new AI attributes):

The Feedback entity is extended with three AI-generated fields (populated automatically on creation). Existing fields are unchanged.

- sentiment: String - AI-classified sentiment; allowed values "Positive", "Neutral", "Negative".
- sentimentScore: Double - confidence/positivity score from 0.0 (very negative) to 1.0 (very positive).
- aiTags: String - comma-separated theme tags extracted from the feedback (e.g. "punctuality,politeness").

AI Service Layer:

GeminiService (service folder):

- embed(text): Returns a semantic float[] embedding via the Gemini embedding model.
- generate(prompt): Returns Gemini-generated text; used for sentiment analysis and summaries.
- isEnabled(): True when a Gemini API key is configured; otherwise callers use a local fallback.

AIService (service folder):

- searchDrivers(query): Ranks drivers for a natural-language query (Feature 1).
- analyzeFeedback(text, category, rating): Returns sentiment, score and tags (Feature 2).
- summarizeDriverFeedback(driverId): Builds the per-driver AI review summary (Feature 3).

Frontend (Angular) Additions:

- services/ai.service.ts - searchDrivers(query) and getDriverFeedbackSummary(driverId); sends the Bearer token in headers and targets /api/ai.
- models/ai.model.ts - DriverFeedbackSummary interface; feedback.model.ts extended with sentiment, sentimentScore and aiTags.
- customerviewdriver component - an "AI Search" box that calls the semantic search endpoint and displays best-match drivers (Feature 1).
- adminviewfeedback component - an AI Sentiment column with colored badges + theme tags, a "Filter by Sentiment" control (Feature 2), and an "AI Review Summary" panel inside the Driver Info modal (Feature 3).

Configuration (application.properties):

Property	Description
gemini.api.key	Google Gemini API key. If empty, AI features fall back to non-AI logic.
gemini.embedding.model	Embedding model id, e.g. text-embedding-004.
gemini.generation.model	Generation model id, e.g. gemini-2.5-flash.
chatbot.similarity.threshold	Minimum cosine similarity for a semantic driver-search match (default 0.55).

Platform Note (Security):

All AI endpoints are grouped under /api/ai/** and are added to permitAll() in the SecurityConfig, consistent with the existing GET /api/driver and GET /api/feedback rules used for auto-evaluation.

Sample AI Integration Screen Shots:

Customer can fetch the Driver via AI Search by prompting

Available Drivers

Search by Driver Name, Vehicle Type, or License...

AI Search: Van driver for school trip Find Drivers Clear

Showing AI best-matches for: "Van driver for school trip"

Driver 003
Vehicle Type: Van
License: Demo12345
Phone: 6789734567
Address: 34, Demo003 Nagar
Experience: 1 years
Hourly Rate: ₹500
Status: Active

[Request](#)

Driver 001
Vehicle Type: SUV
License: 987654Test
Phone: 9876543211
Address: 34, Demo Nagar
Experience: 3 years
Hourly Rate: ₹800
Status: Active

[Request](#)

Driver 002
Vehicle Type: Sedan
License: Test123456
Phone: 9876234523
Address: 34, demo02 Nagar
Experience: 2 years
Hourly Rate: ₹400
Status: Active

[Request](#)

Admin can filter the Feedback by AI Sentiment analysis – received by customer

Feedback Received

Filter by Category: All Categories Filter by Sentiment: All Sentiments, Positive, Neutral, Negative

S.NO	USER NAME	FEEDBACK	AI SENTIMENT	POSTED DATE	ACTION
1	Customer1	Good Punctuality - Daily morning for school trip	Positive punctuality	26/07/2026	Show Profile View Driver Info

Admin/Customer can view the AI Review Summary – Received from the customer

Driver Details

Driver Name: Driver 003
License Number: Demo12345
Experience Years: 1
Vehicle Type: Van
Hourly Rate: ₹500

AI Review Summary

1 review(s) · Avg rating 4/5 · AI score 80/100

This driver consistently demonstrates good punctuality, especially for daily morning school trips. The customer appreciates the reliable service.

Strengths

- Good punctuality
- Consistent service